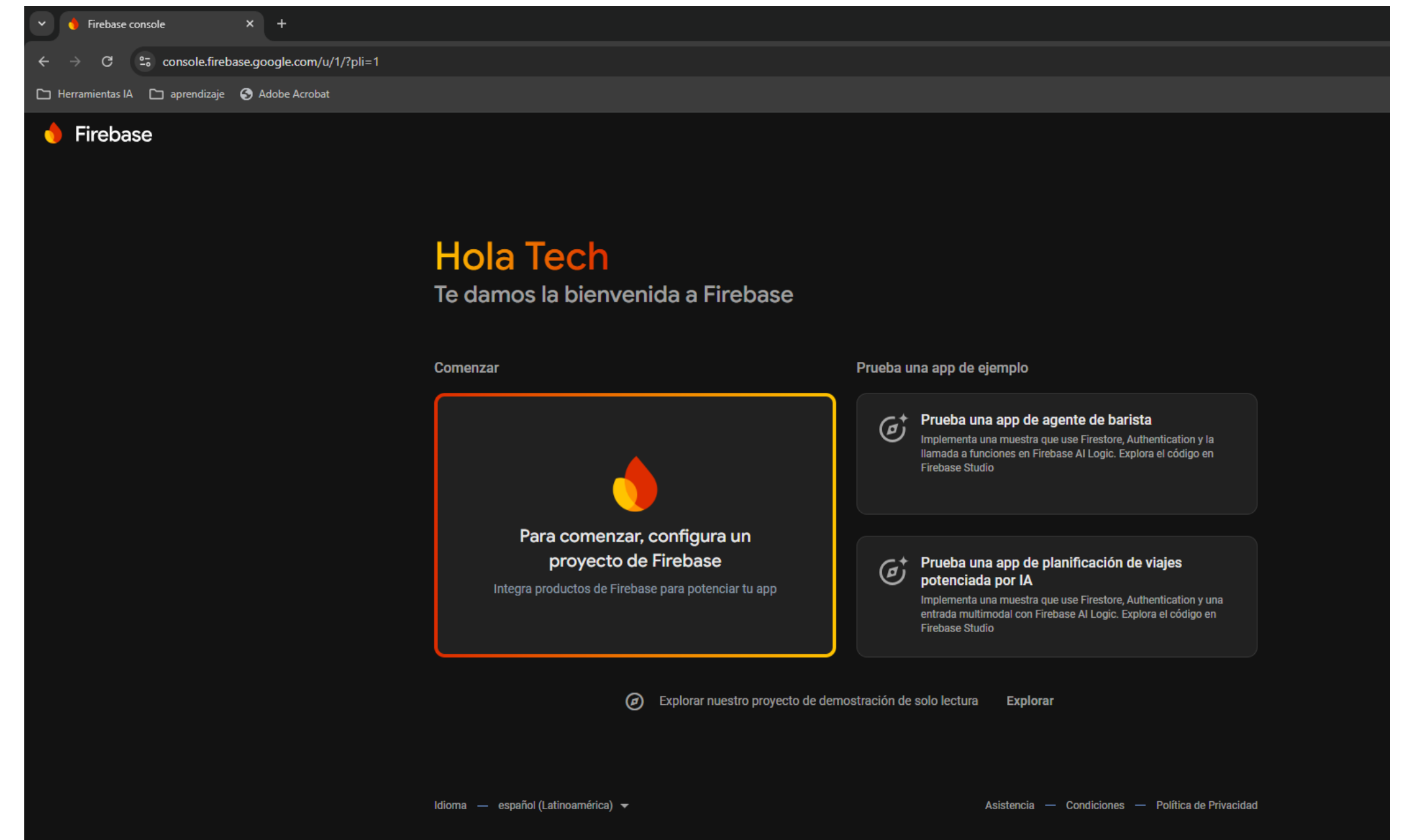


**Web y Patrones
Semana 4**

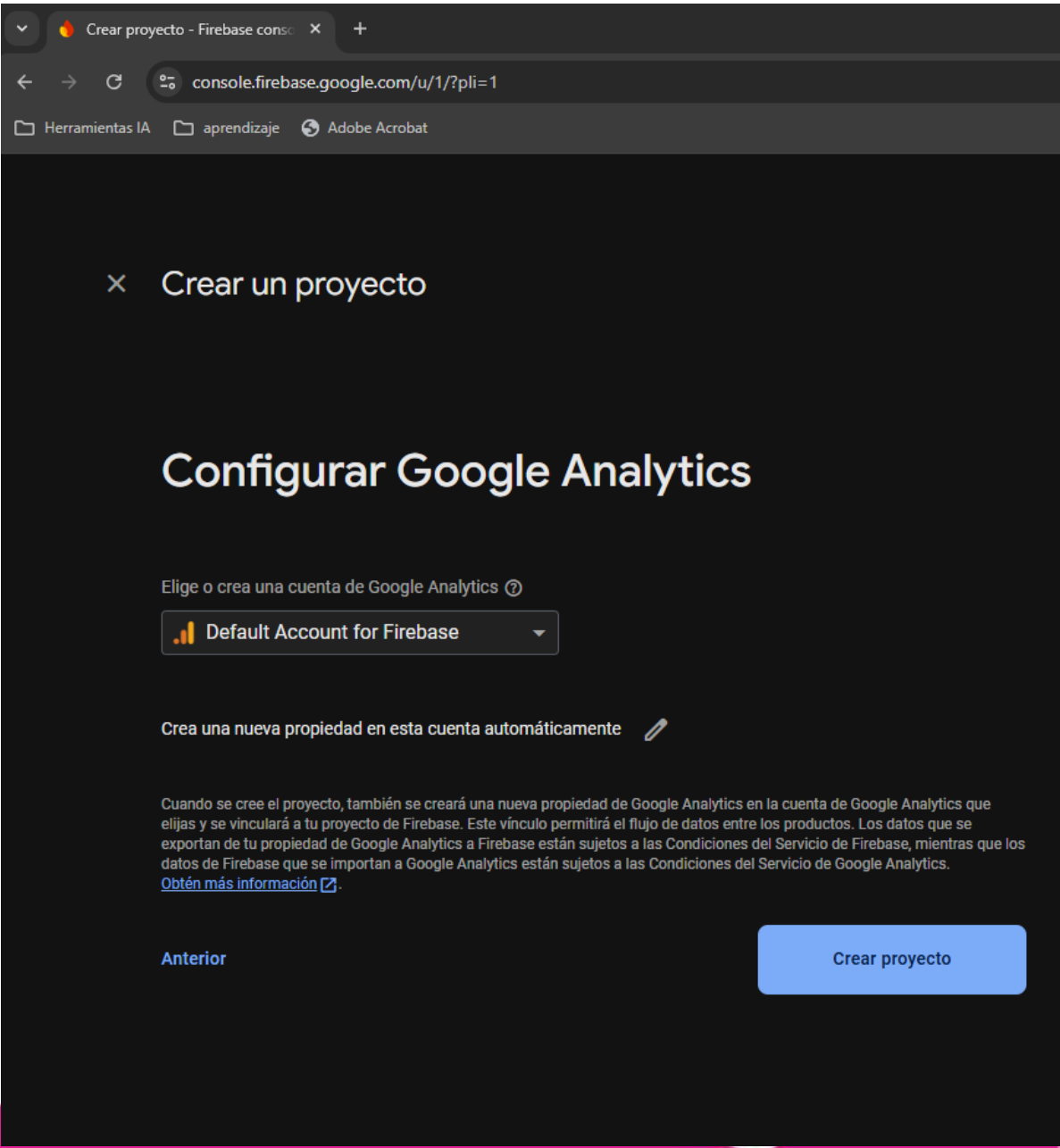
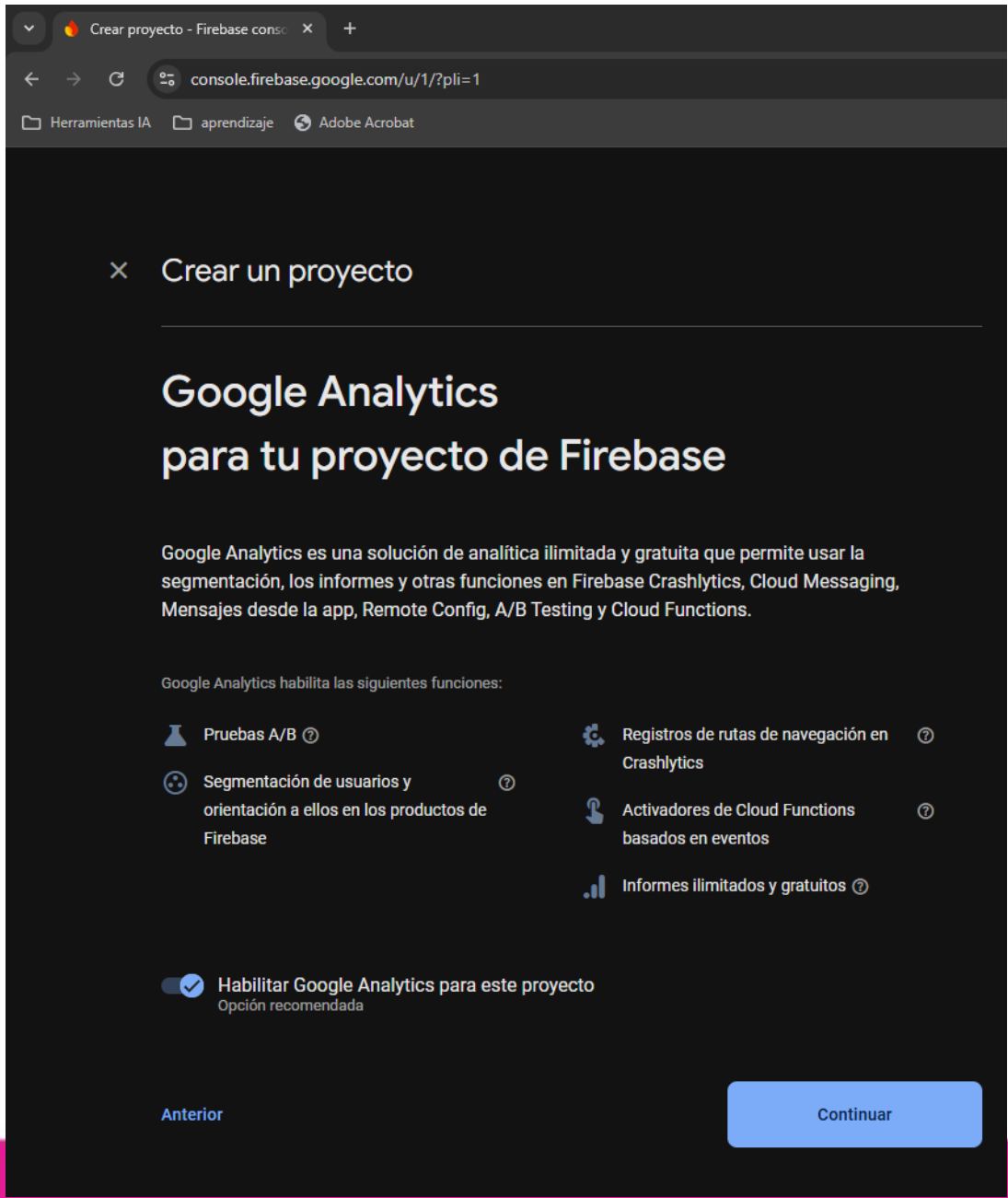
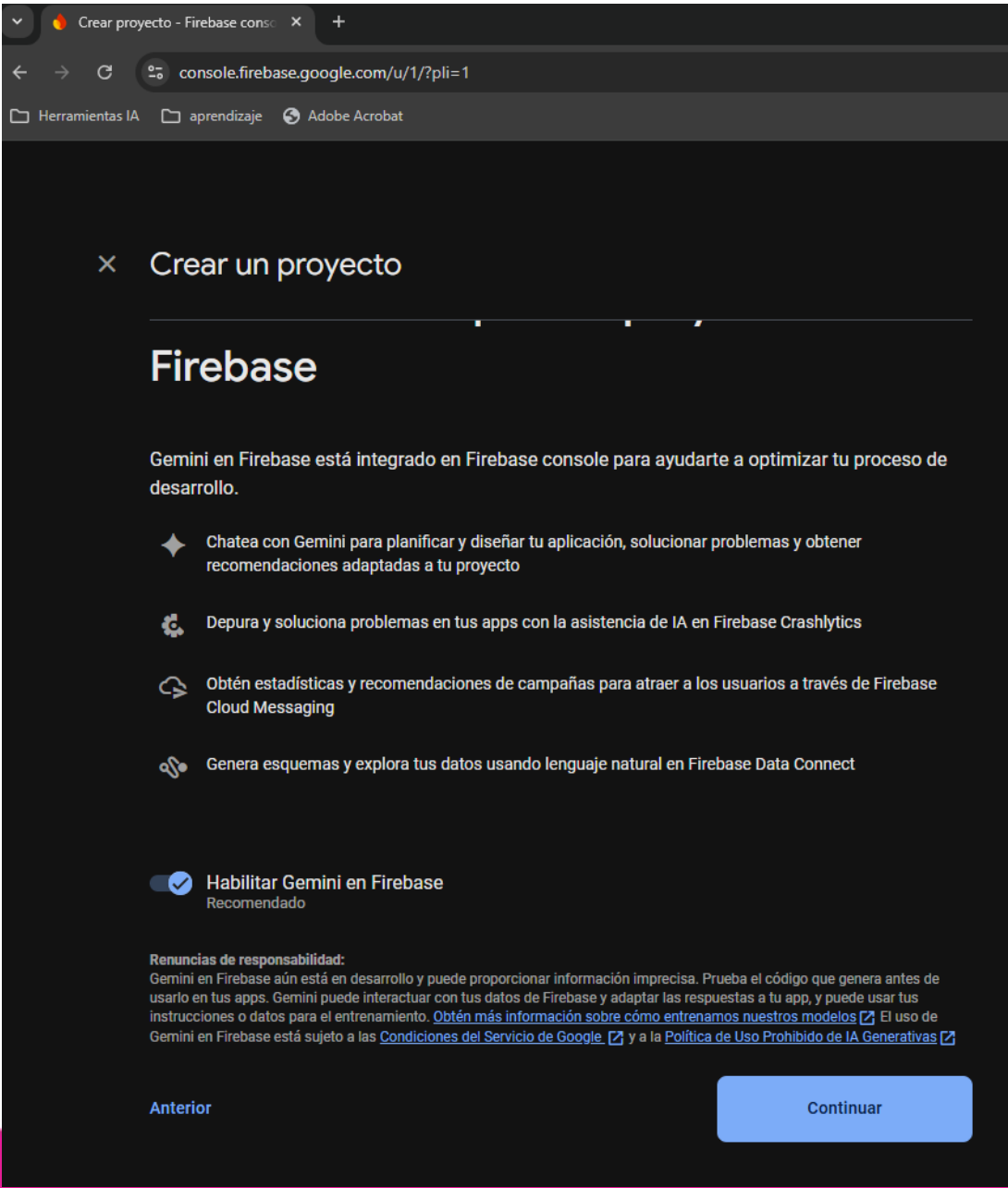
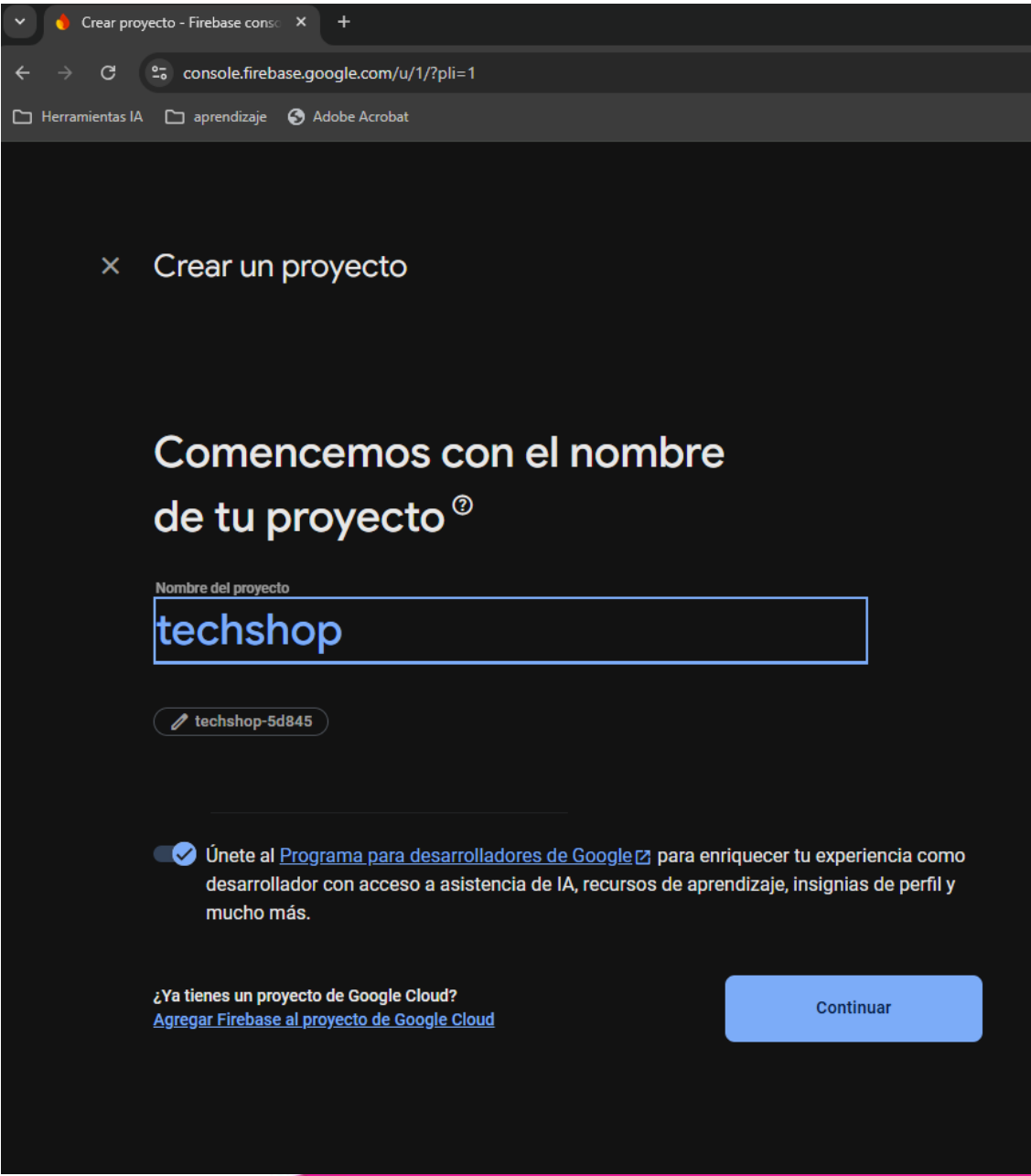
Firestore y Crud

- Creamos nuestra cuenta en Firebase para poder utilizar el espacio “storage” en el almacenamiento de imágenes de nuestro sitio web
- Se ingresa a <https://console.firebase.google.com/> mediante algún usuario de Google.

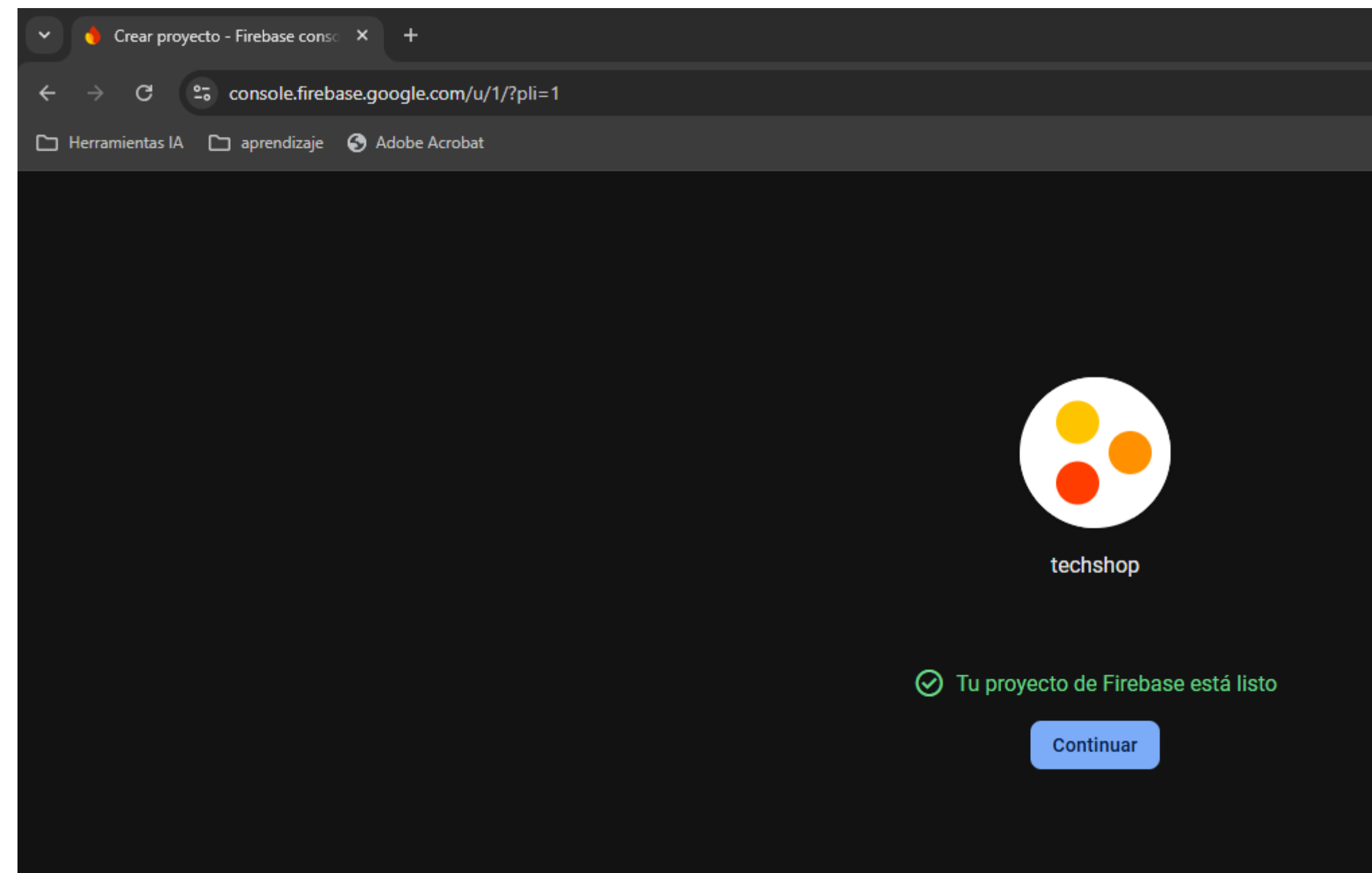


Firebase y CRUD

- Se crea el proyecto techshop.

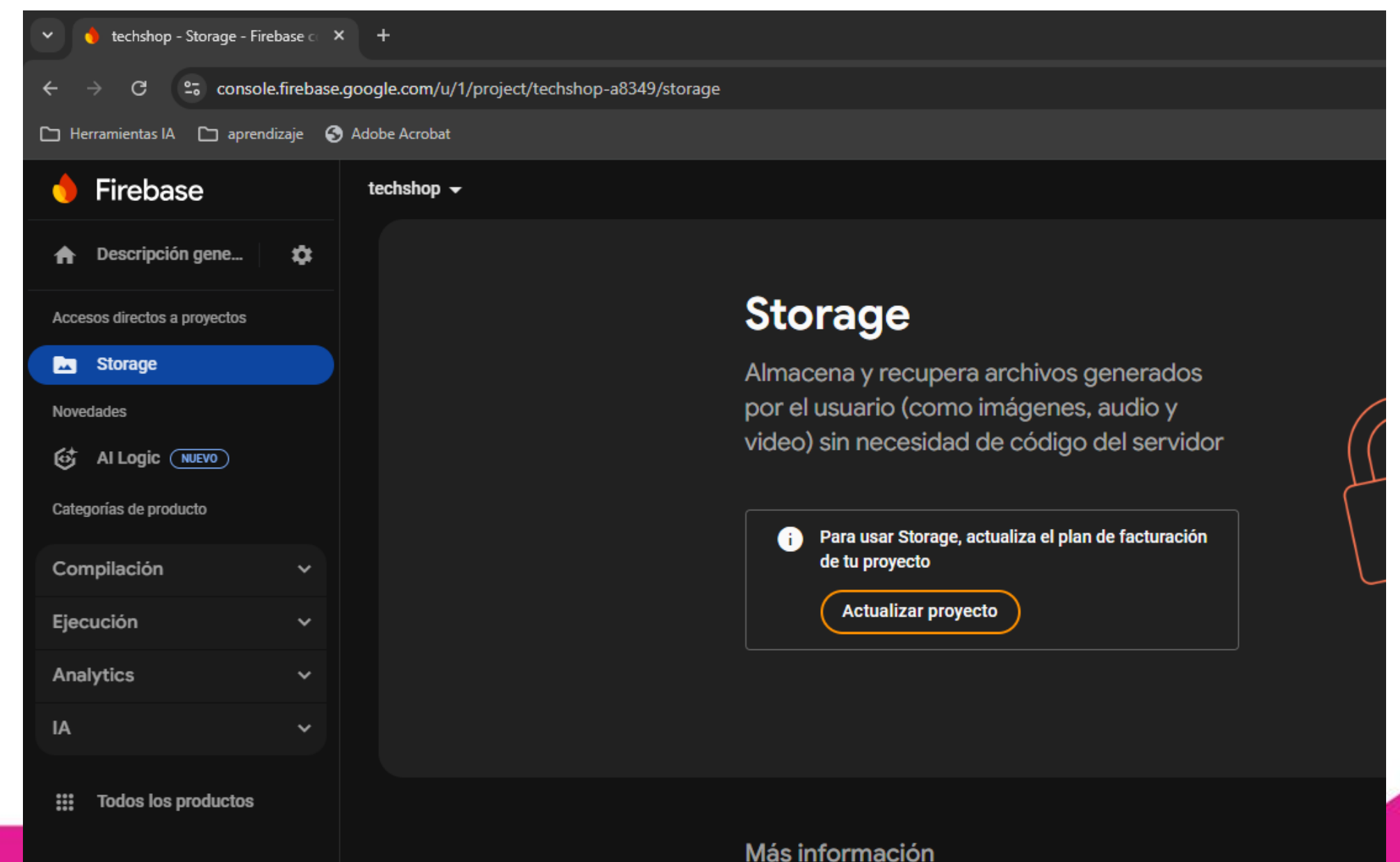
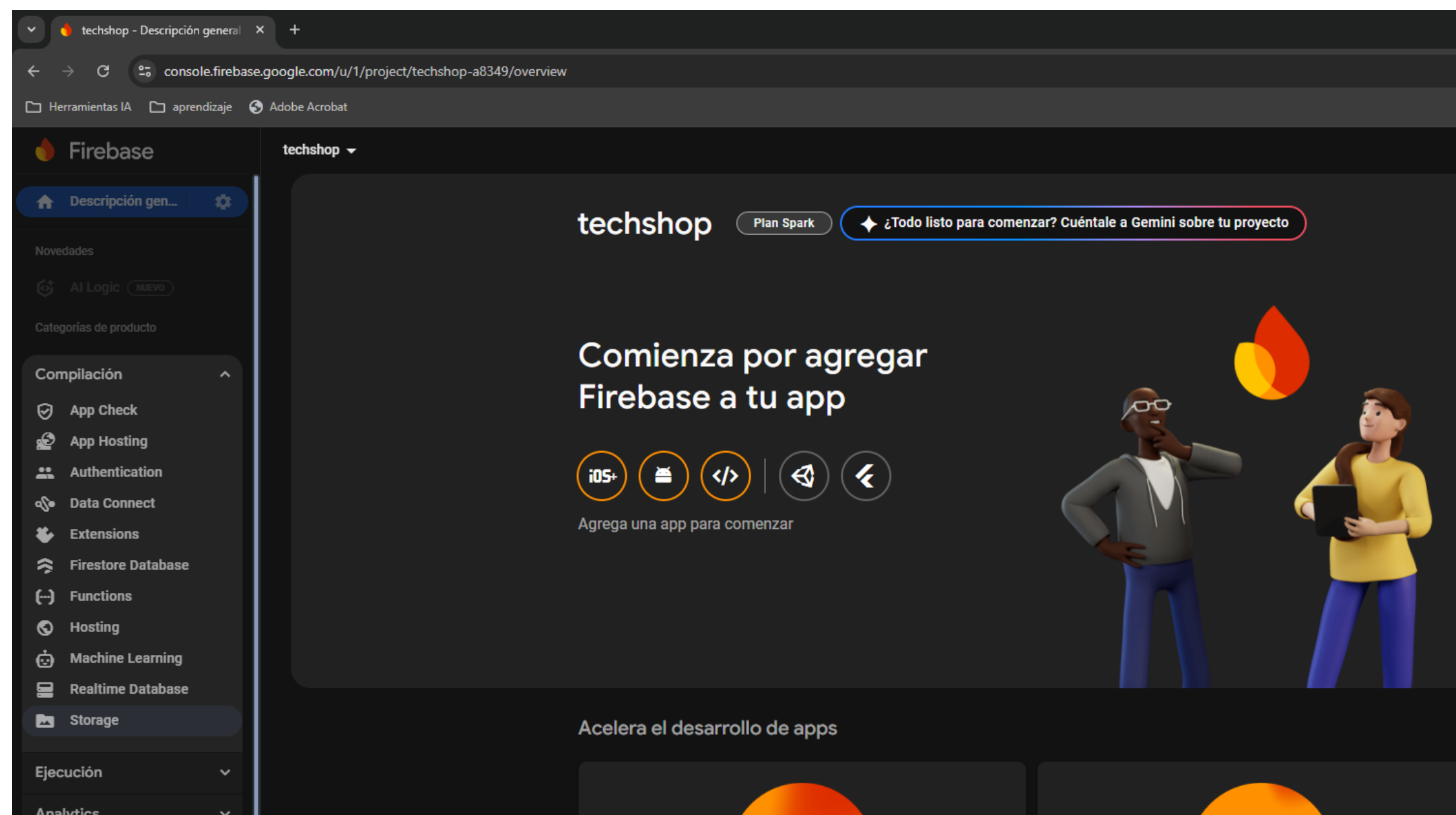


Firestore y CRUD



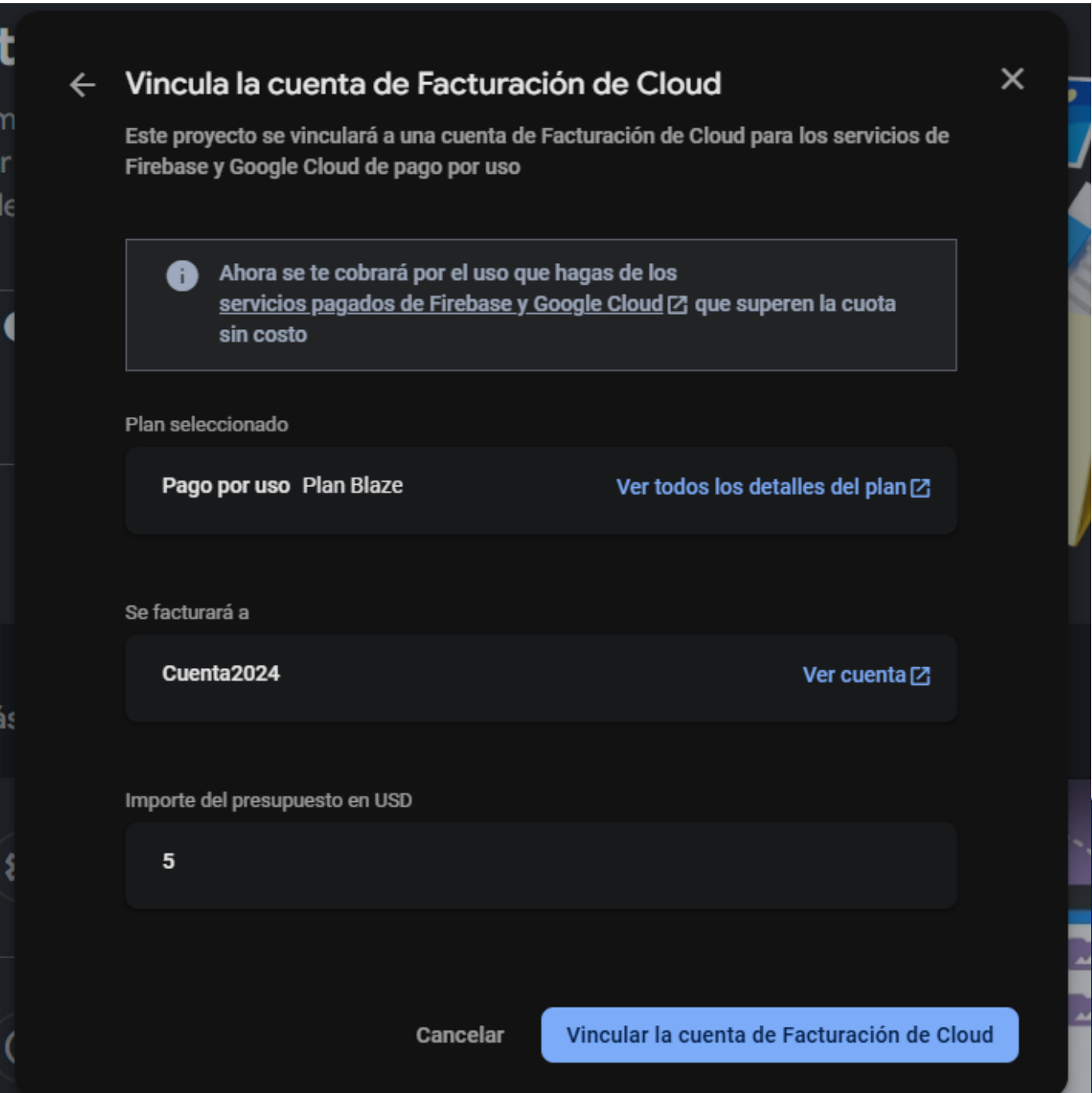
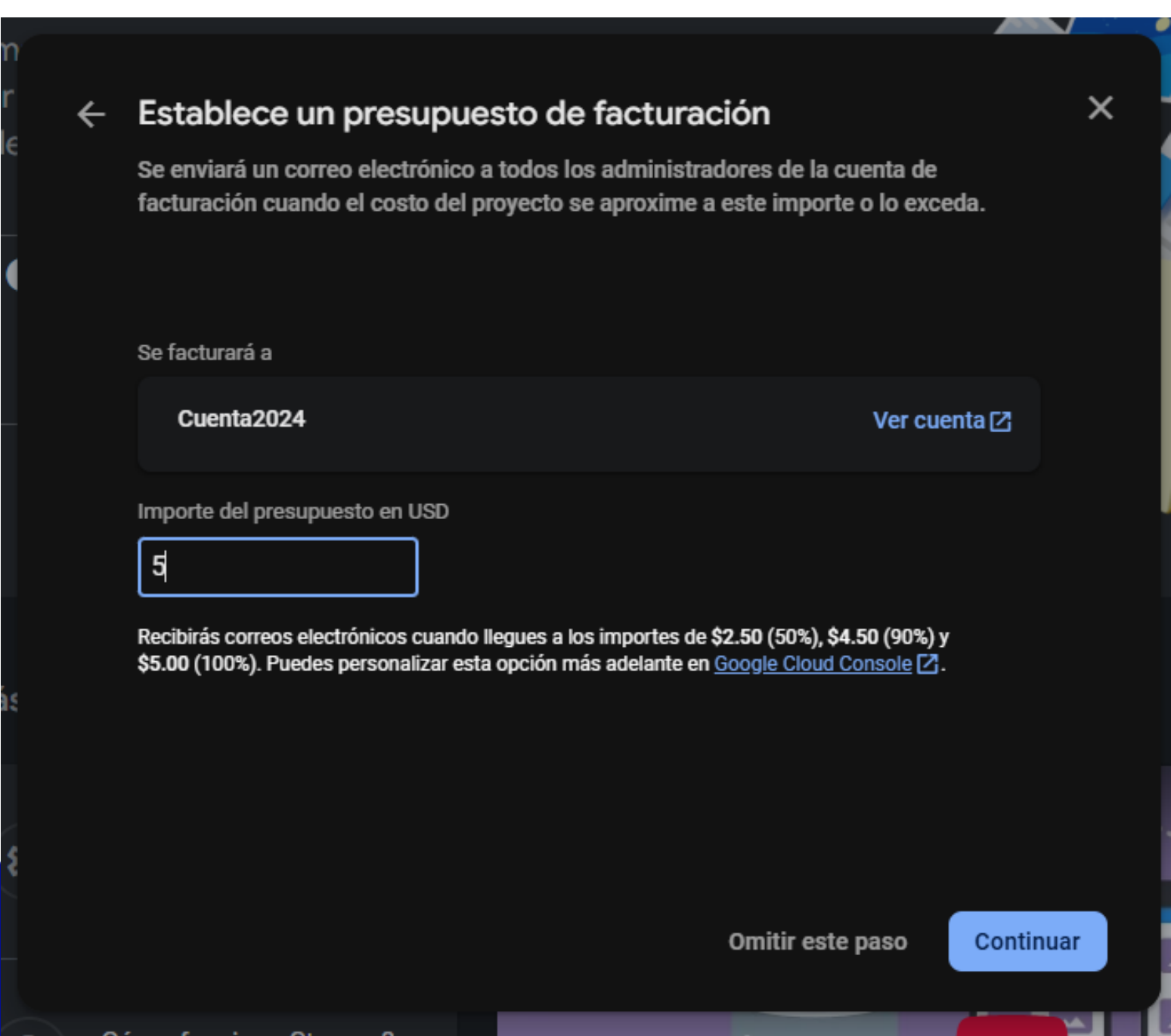
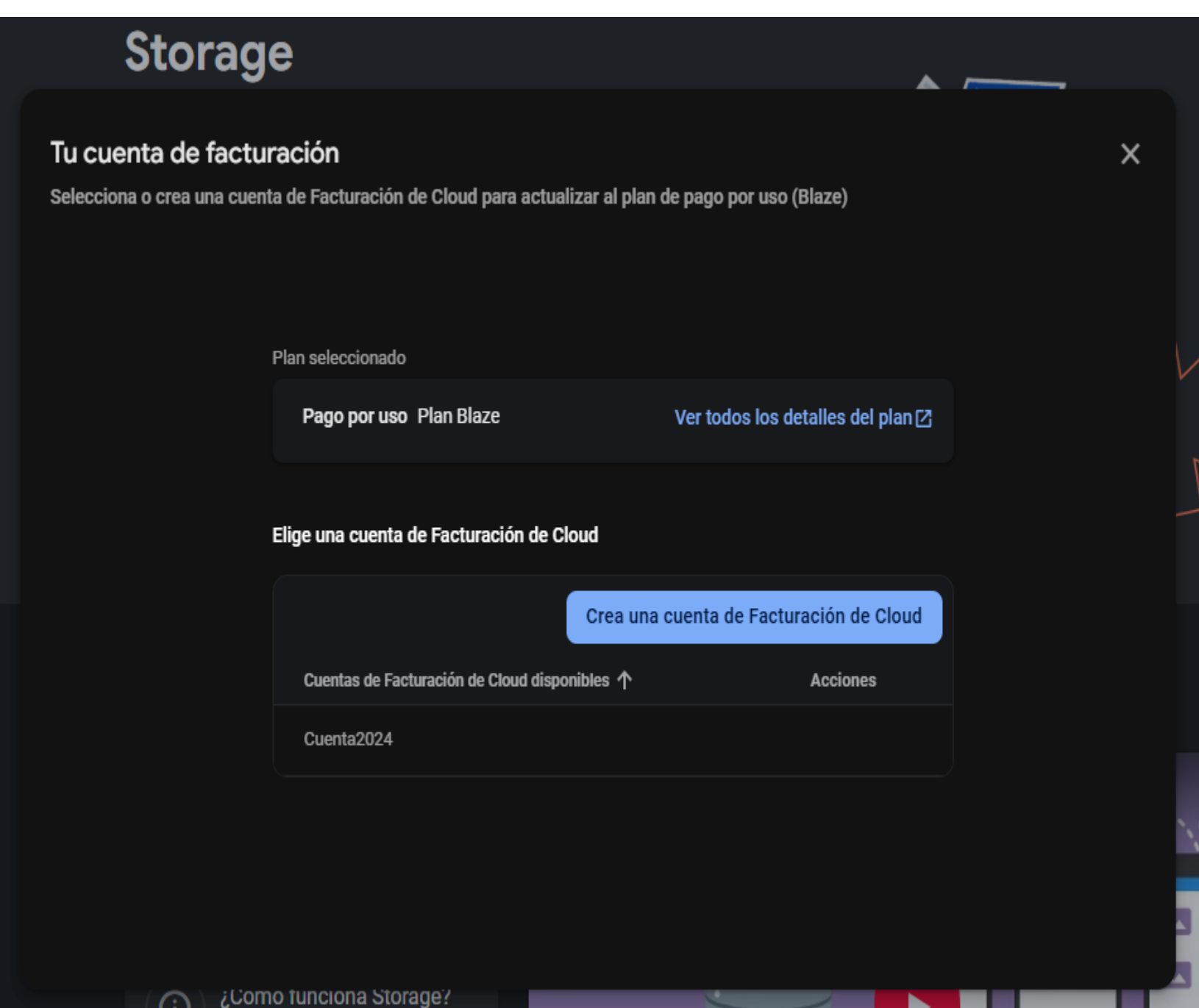
Se habilita el Storage

En la sección de “Compilación” se da click en Storage.
Hay que activar el plan de facturación (no se genera un cobro)



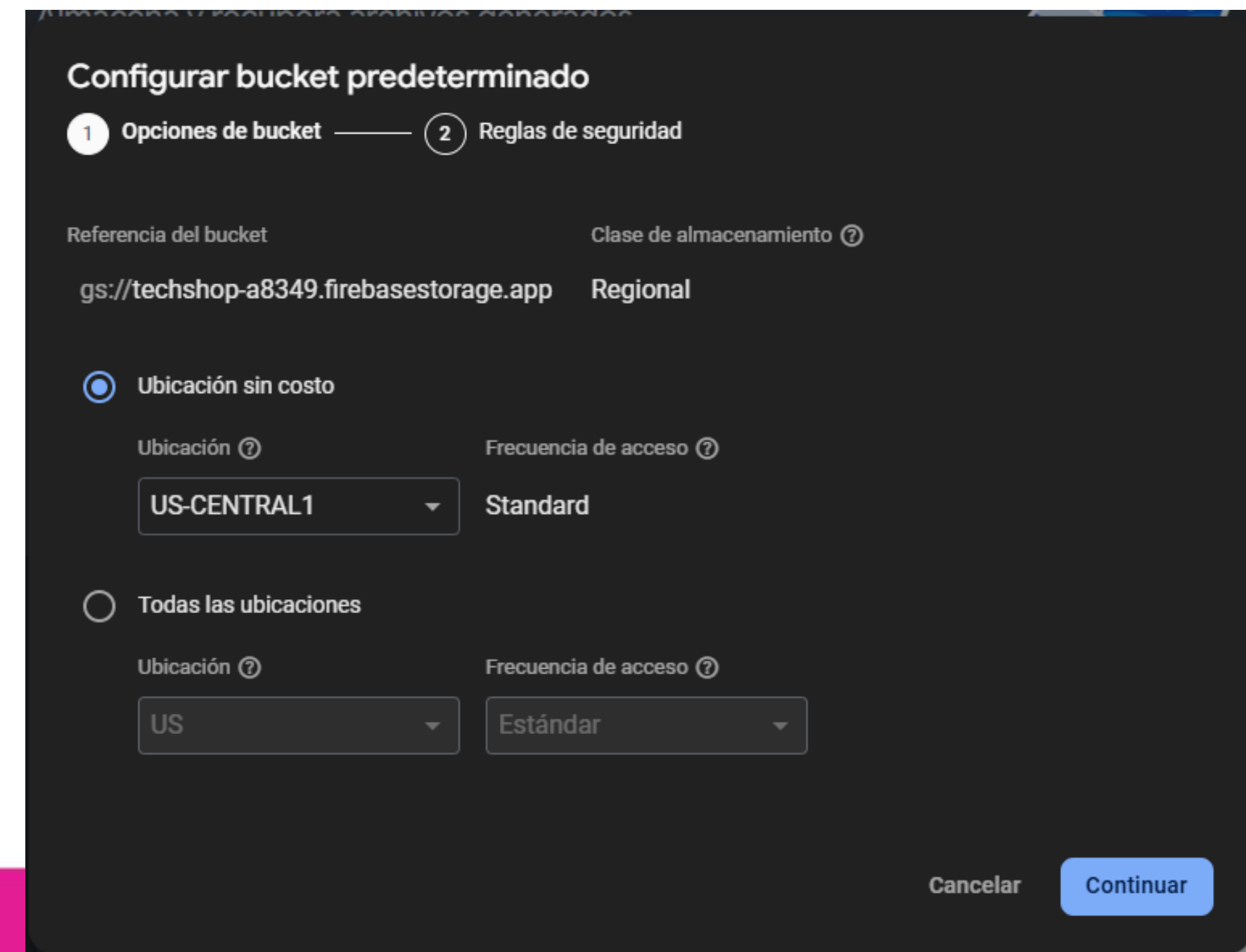
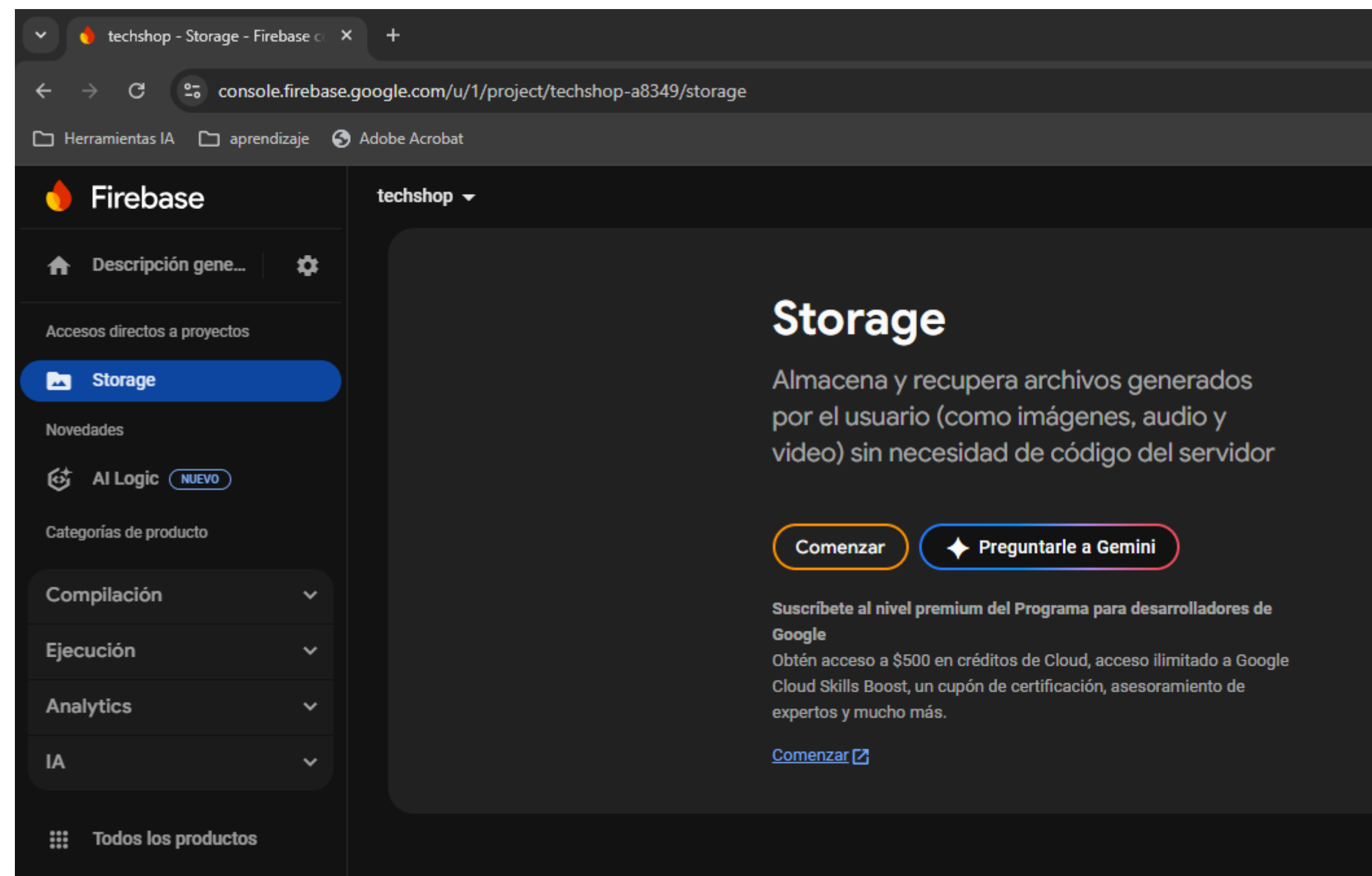
Cuenta de Facturación

Como Firebase se utiliza a nivel comercial, la creación de la cuenta de facturación es un medio que utiliza Google para recuperar posible clientes comerciales. Lo que se realizará en el curso no genera ningún cargo.



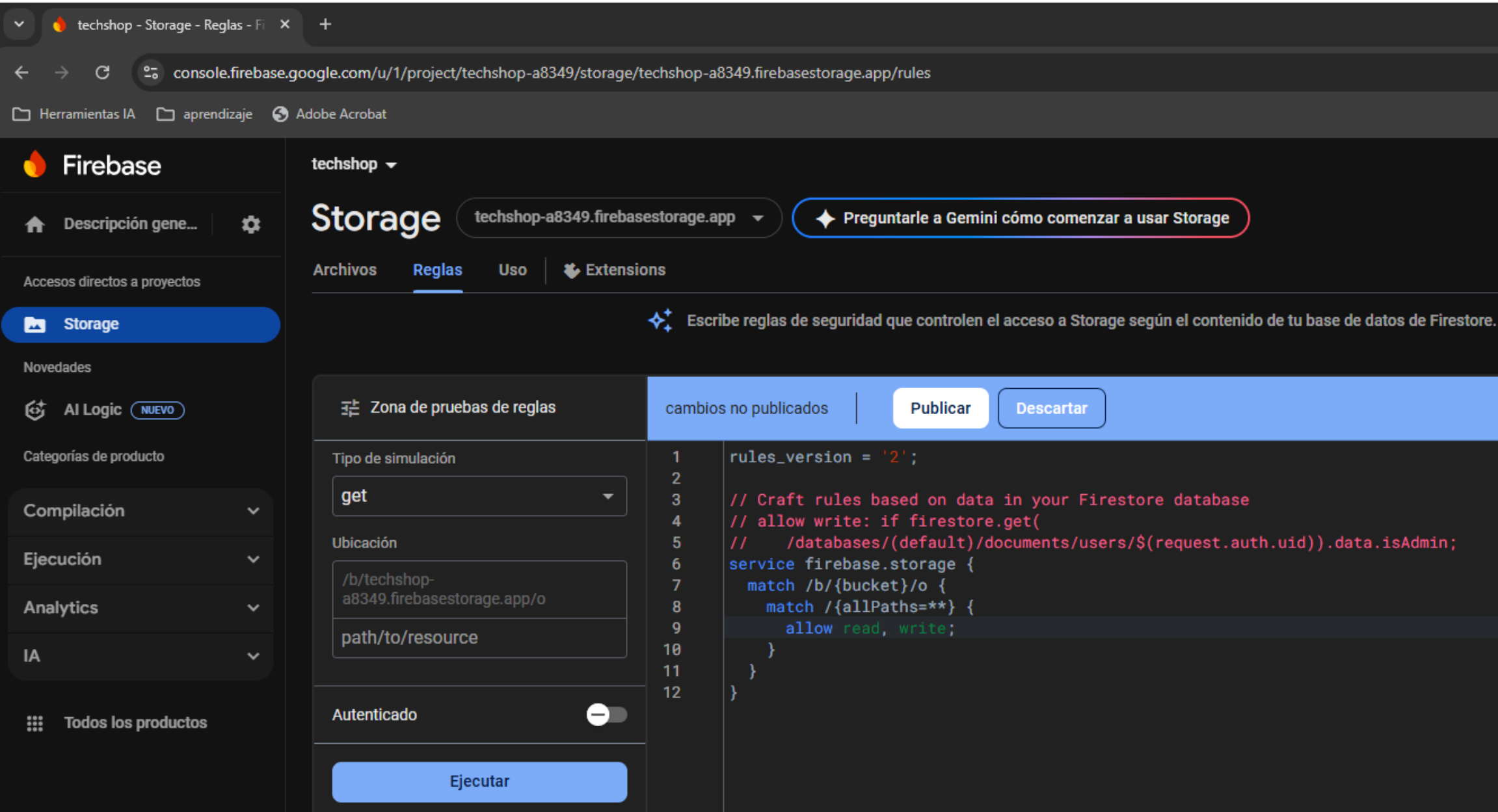
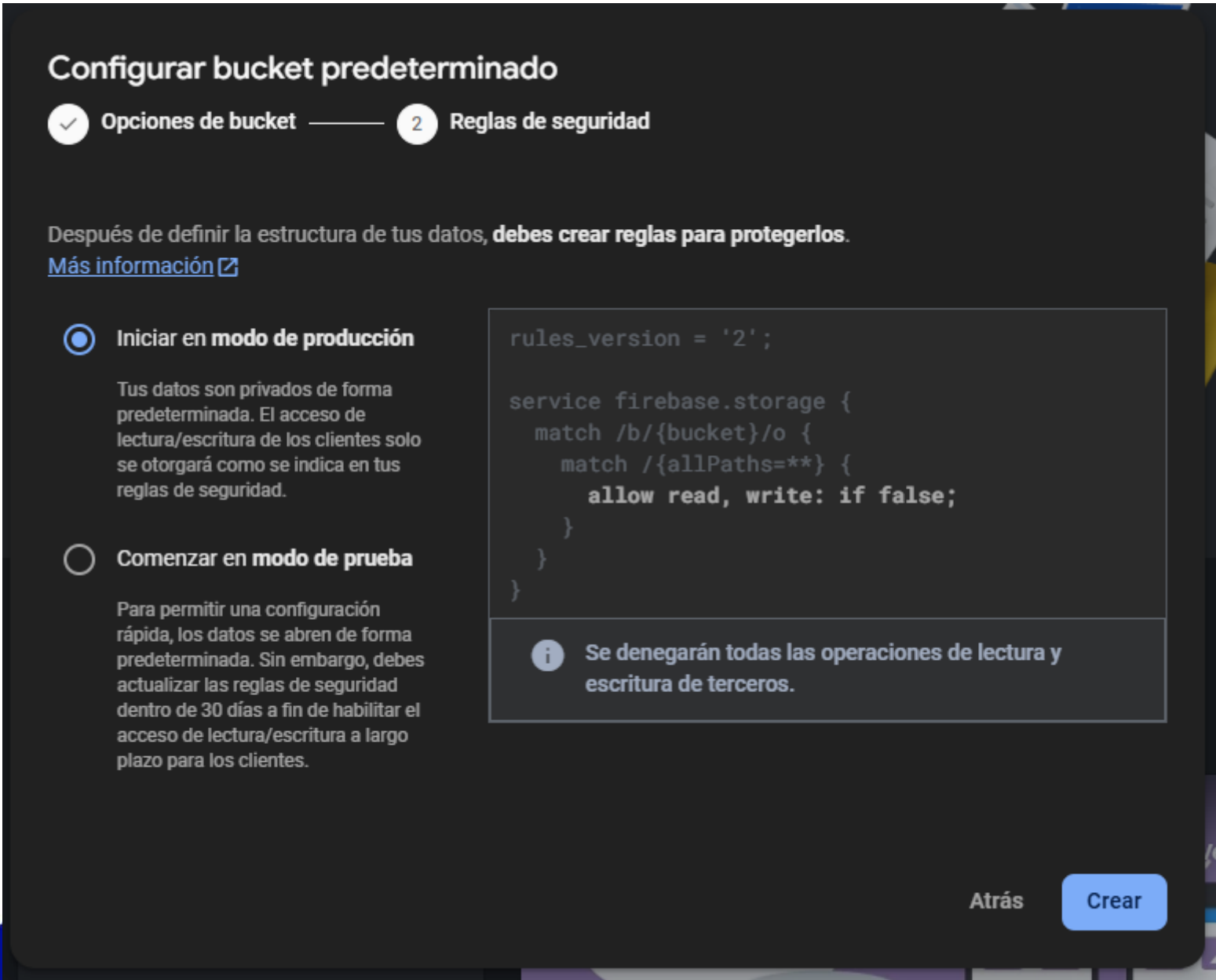
Comenzar y configurar bucket sin costo

Se da la opción de comenzar el Storage y si utiliza una ubicación sin costo.



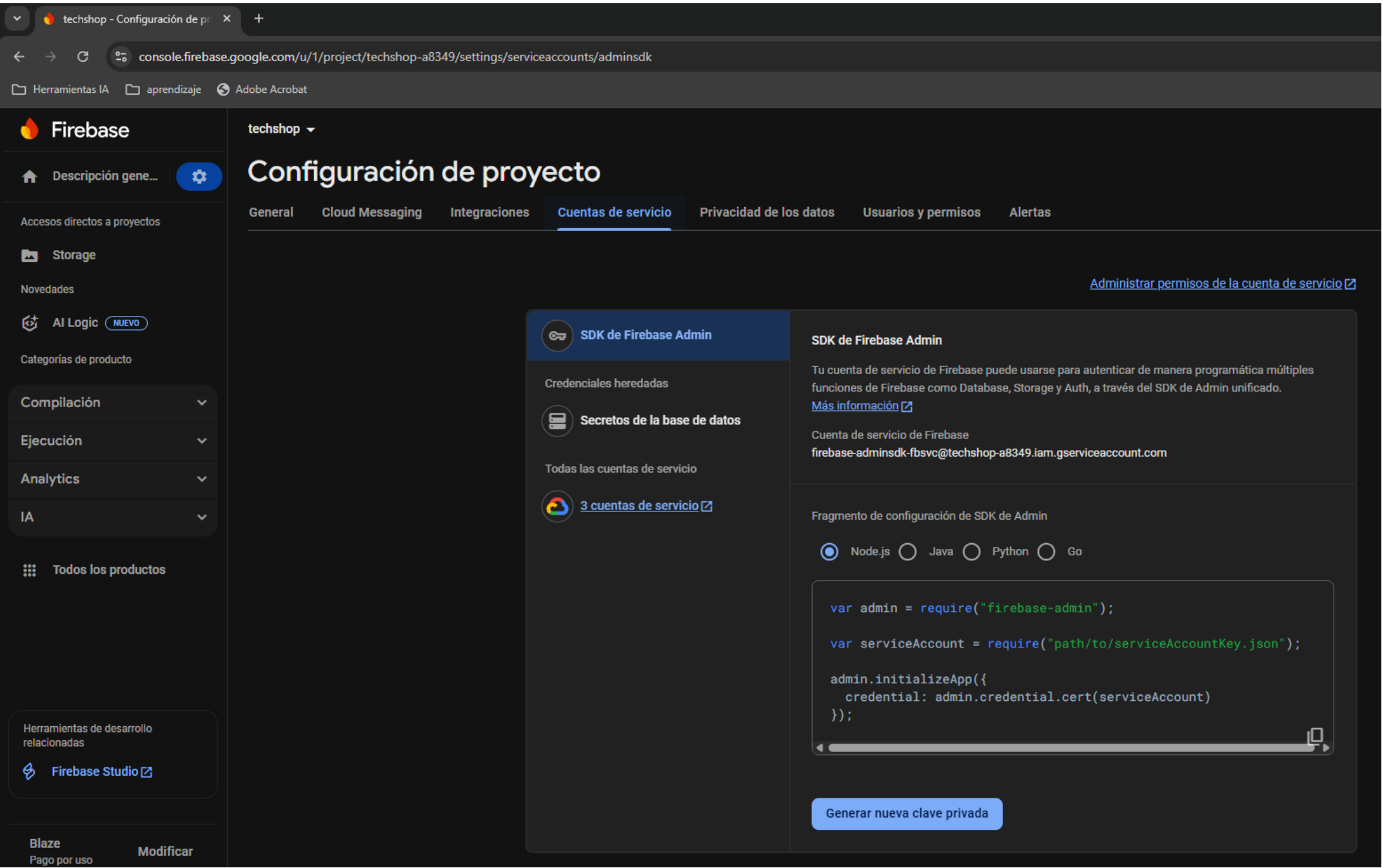
Definición de reglas públicas

Se deja la opción de iniciar modo de producción y luego se hacen públicas las reglas. Observen que se modifica la línea 9 de las reglas. Originalmente estaba “allow read, write: if false” y queda “allow read, write”.



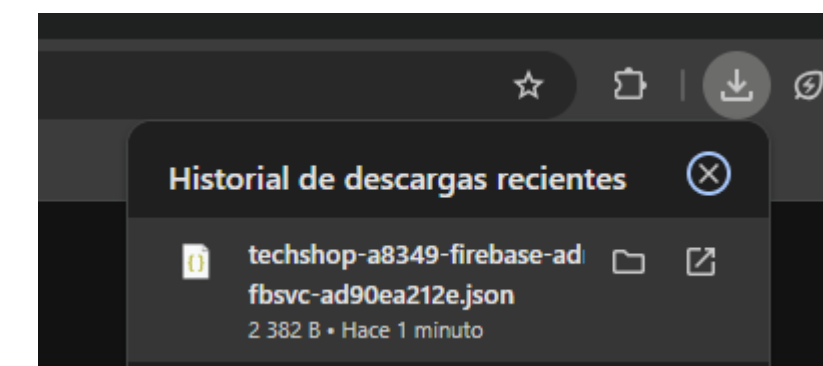
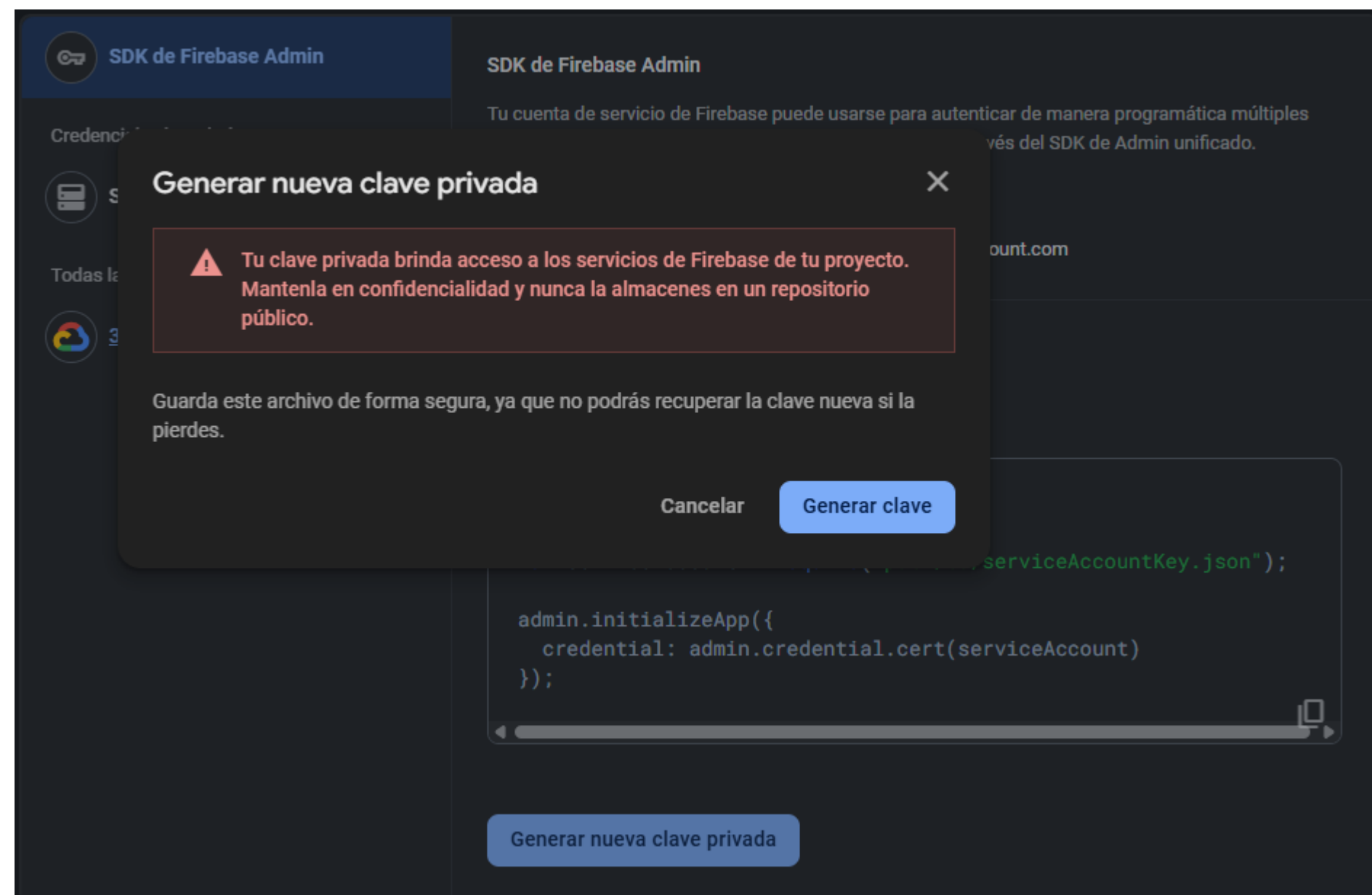
Generación de clave privada

Se genera una clave privada para colocar en el proyecto tienda



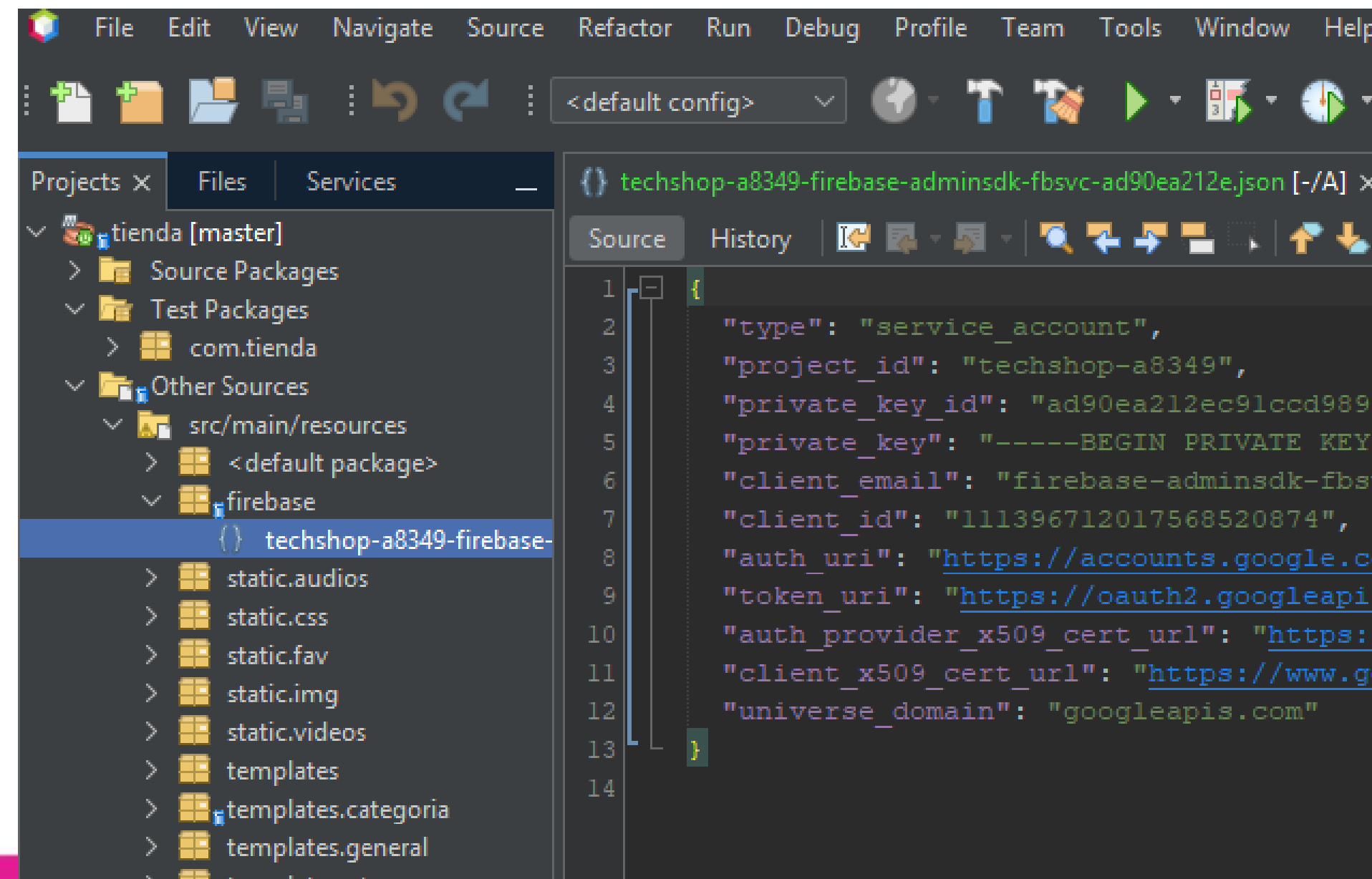
Generación y descarga

Se genera la clave privada y se debe descargar dentro del proyecto tienda, debe ser un lugar seguro



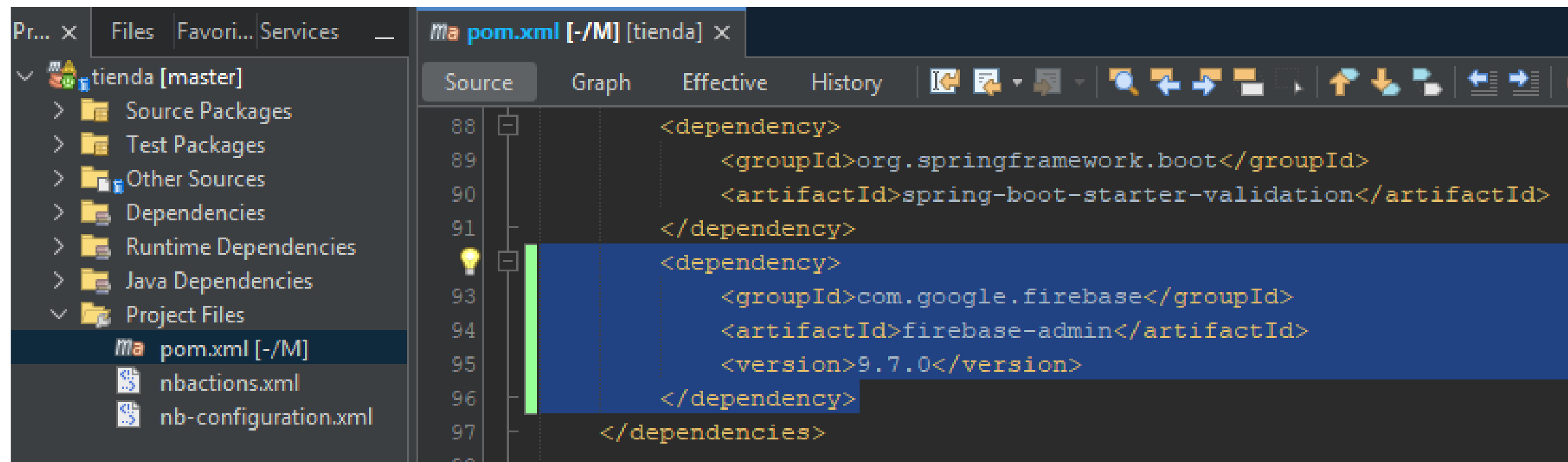
Folder firebase

Se crea un nuevo folder en la raíz de “default package”, llamado “Firebase”, ahí, colocamos el archivo descargado.



pom.xml

Se incluye la dependencia de Firebase en el archivo pom.xml, recuerde hacer un “Clean and Build” (martillo y escoba)

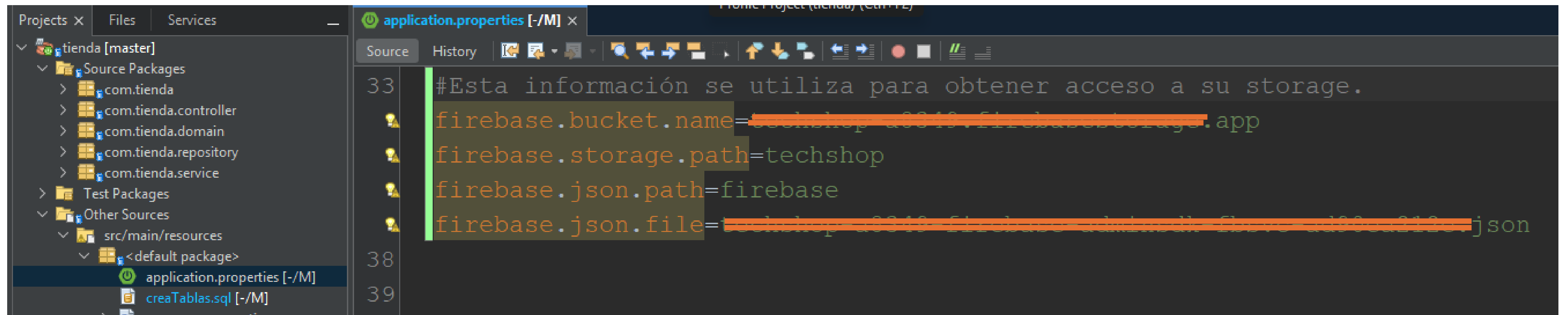
A screenshot of an IDE window showing the 'pom.xml' file. The left sidebar shows a project tree with 'tienda [master]' expanded, and 'pom.xml [-/M]' selected under 'Project Files'. The main editor shows the XML content of 'pom.xml'. The 'dependencies' section is visible, with a new dependency for 'firebase-admin' added. The dependency is highlighted with a blue background. The XML code is as follows:

```
88 <dependency>
89   <groupId>org.springframework.boot</groupId>
90   <artifactId>spring-boot-starter-validation</artifactId>
91 </dependency>
92
93 <dependency>
94   <groupId>com.google.firebase</groupId>
95   <artifactId>firebase-admin</artifactId>
96   <version>9.7.0</version>
97 </dependency>
98 </dependencies>
```

Recuerda: “Martillo y Escoba”

FirestoreStorageService

Agreguemos a application.properties 4 valores que se utilizarán en la comunicación con firebase

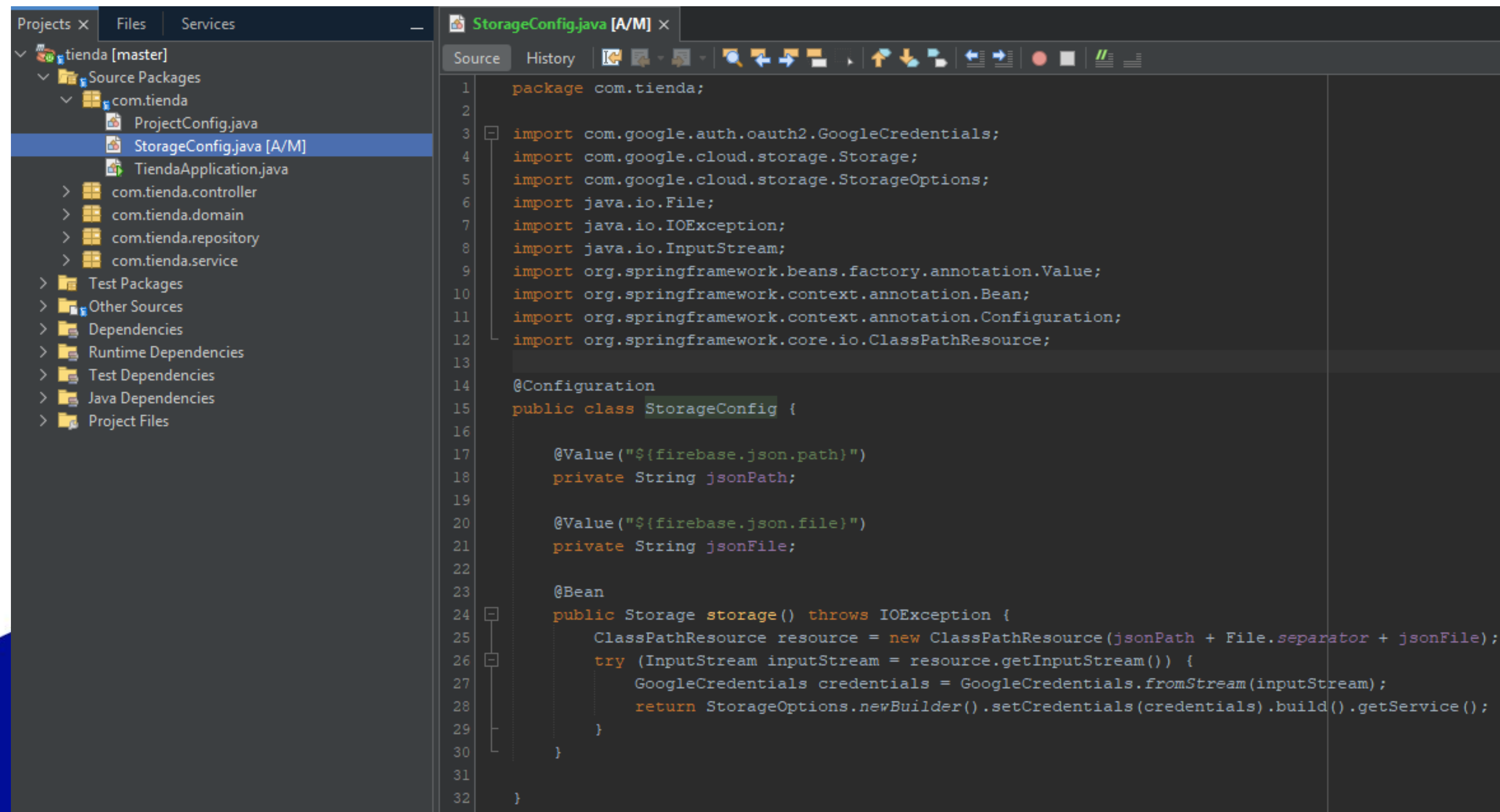


The screenshot shows an IDE window with the file `application.properties` open. The left sidebar shows a project structure for `tienda [master]` with packages like `com.tienda`, `com.tienda.controller`, `com.tienda.domain`, `com.tienda.repository`, `com.tienda.service`, `Test Packages`, and `Other Sources`. The main editor shows the following code:

```
33 #Esta información se utiliza para obtener acceso a su storage.  
34 firebase.bucket.name=techshop-43343-firebasestorage.app  
35 firebase.storage.path=techshop  
36 firebase.json.path=firebase  
37 firebase.json.file=techshop-43343-firebase-adminsdk-fb0vs-ad90ca2123.json  
38  
39
```


FirestoreStorageService

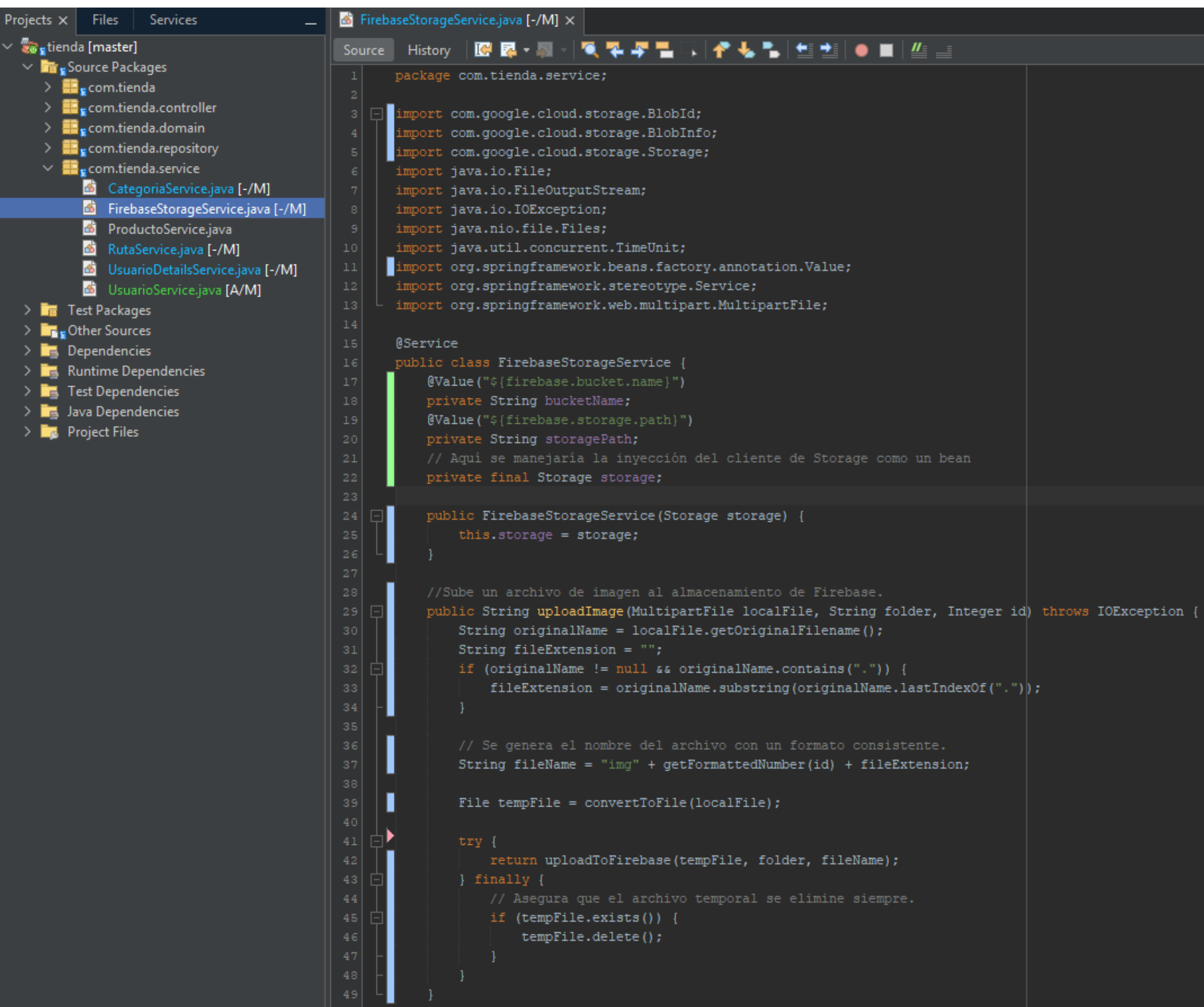
Se crea una nueva clase StorageConfig para gestionar las credenciales del Storage



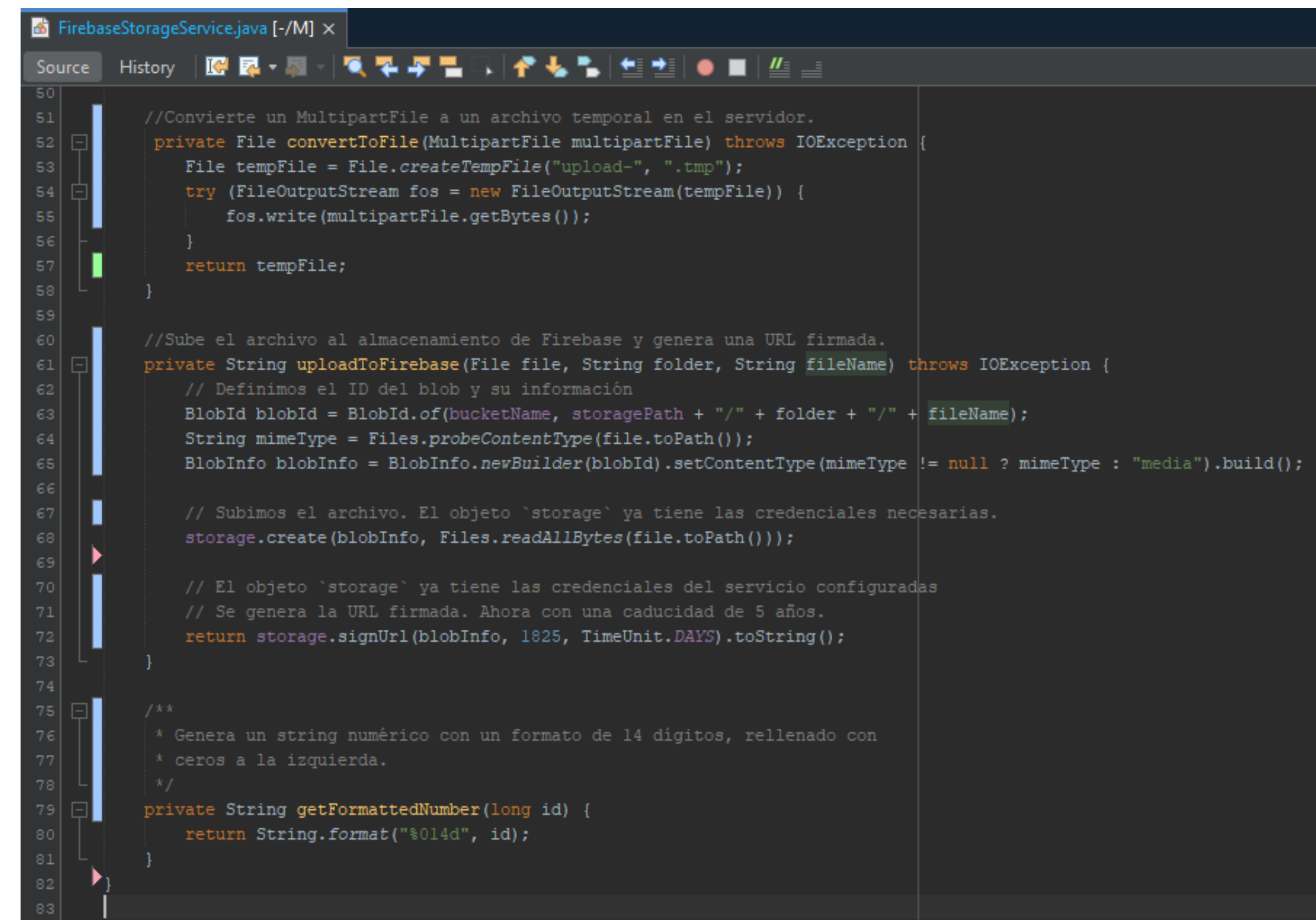
```
1 package com.tienda;
2
3 import com.google.auth.oauth2.GoogleCredentials;
4 import com.google.cloud.storage.Storage;
5 import com.google.cloud.storage.StorageOptions;
6 import java.io.File;
7 import java.io.IOException;
8 import java.io.InputStream;
9 import org.springframework.beans.factory.annotation.Value;
10 import org.springframework.context.annotation.Bean;
11 import org.springframework.context.annotation.Configuration;
12 import org.springframework.core.io.ClassPathResource;
13
14 @Configuration
15 public class StorageConfig {
16
17     @Value("${firebase.json.path}")
18     private String jsonPath;
19
20     @Value("${firebase.json.file}")
21     private String jsonFile;
22
23     @Bean
24     public Storage storage() throws IOException {
25         ClassPathResource resource = new ClassPathResource(jsonPath + File.separator + jsonFile);
26         try (InputStream inputStream = resource.getInputStream()) {
27             GoogleCredentials credentials = GoogleCredentials.fromStream(inputStream);
28             return StorageOptions.newBuilder().setCredentials(credentials).build().getService();
29         }
30     }
31 }
32 }
```

FirebaseStorageService

Descargue de los recurso la clase FirebaseStorageService.java, los ajustes se realizan a application.properties



```
1 package com.tienda.service;
2
3 import com.google.cloud.storage.BlobId;
4 import com.google.cloud.storage.BlobInfo;
5 import com.google.cloud.storage.Storage;
6 import java.io.File;
7 import java.io.FileOutputStream;
8 import java.io.IOException;
9 import java.nio.file.Files;
10 import java.util.concurrent.TimeUnit;
11 import org.springframework.beans.factory.annotation.Value;
12 import org.springframework.stereotype.Service;
13 import org.springframework.web.multipart.MultipartFile;
14
15 @Service
16 public class FirebaseStorageService {
17     @Value("${firebase.bucket.name}")
18     private String bucketName;
19     @Value("${firebase.storage.path}")
20     private String storagePath;
21     // Aquí se manejaría la inyección del cliente de Storage como un bean
22     private final Storage storage;
23
24     public FirebaseStorageService(Storage storage) {
25         this.storage = storage;
26     }
27
28     //Sube un archivo de imagen al almacenamiento de Firebase.
29     public String uploadImage(MultipartFile localFile, String folder, Integer id) throws IOException {
30         String originalName = localFile.getOriginalFilename();
31         String fileExtension = "";
32         if (originalName != null && originalName.contains(".")) {
33             fileExtension = originalName.substring(originalName.lastIndexOf("."));
34         }
35
36         // Se genera el nombre del archivo con un formato consistente.
37         String fileName = "img" + getFormattedNumber(id) + fileExtension;
38
39         File tempFile = convertToFile(localFile);
40
41         try {
42             return uploadToFirebase(tempFile, folder, fileName);
43         } finally {
44             // Asegura que el archivo temporal se elimine siempre.
45             if (tempFile.exists()) {
46                 tempFile.delete();
47             }
48         }
49     }
50 }
```



```
50
51 //Convierte un MultipartFile a un archivo temporal en el servidor.
52 private File convertToFile(MultipartFile multipartFile) throws IOException {
53     File tempFile = File.createTempFile("upload-", ".tmp");
54     try (FileOutputStream fos = new FileOutputStream(tempFile)) {
55         fos.write(multipartFile.getBytes());
56     }
57     return tempFile;
58 }
59
60 //Sube el archivo al almacenamiento de Firebase y genera una URL firmada.
61 private String uploadToFirebase(File file, String folder, String fileName) throws IOException {
62     // Definimos el ID del blob y su información
63     BlobId blobId = BlobId.of(bucketName, storagePath + "/" + folder + "/" + fileName);
64     String mimeType = Files.probeContentType(file.toPath());
65     BlobInfo blobInfo = BlobInfo.newBuilder(blobId).setContentType(mimeType != null ? mimeType : "media").build();
66
67     // Subimos el archivo. El objeto `storage` ya tiene las credenciales necesarias.
68     storage.create(blobInfo, Files.readAllBytes(file.toPath()));
69
70     // El objeto `storage` ya tiene las credenciales del servicio configuradas
71     // Se genera la URL firmada. Ahora con una caducidad de 5 años.
72     return storage.signUrl(blobInfo, 1825, TimeUnit.DAYS).toString();
73 }
74
75 /**
76  * Genera un string numérico con un formato de 14 dígitos, relleno con
77  * ceros a la izquierda.
78  */
79 private String getFormattedNumber(long id) {
80     return String.format("%014d", id);
81 }
82
83 }
```

CategoriaService.java

Se agregan a la clase, los
métodos: get, save y delete

```
CategoriaService.java [-/M] x
Source History
33 @Transactional(readOnly = true)
34 public Optional<Categoria> getCategoria(Integer idCategoria) {
35     return categoriaRepository.findById(idCategoria);
36 }
37
38 @Transactional
39 public void save(Categoria categoria, MultipartFile imagenFile) {
40     categoria = categoriaRepository.save(categoria);
41     if (!imagenFile.isEmpty()) { //Si no está vacío... pasaron una imagen...
42         try {
43             String rutaImagen = firebaseStorageService.uploadImage(
44                 imagenFile, "categoria",
45                 categoria.getIdCategoria());
46             categoria.setRutaImagen(rutaImagen);
47             categoriaRepository.save(categoria);
48         } catch (IOException e) {
49
50         }
51     }
52 }
53
54 @Transactional
55 public void delete(Integer idCategoria) {
56     // Verifica si la categoría existe antes de intentar eliminarlo
57     if (!categoriaRepository.existsById(idCategoria)) {
58         // Lanza una excepción para indicar que el usuario no fue encontrado
59         throw new IllegalArgumentException("La categoría con ID " + idCategoria + " no existe.");
60     }
61     try {
62         categoriaRepository.deleteById(idCategoria);
63     } catch (DataIntegrityViolationException e) {
64         // Lanza una nueva excepción para encapsular el problema de integridad de datos
65         throw new IllegalStateException("No se puede eliminar la categoría. Tiene datos asociados.", e);
66     }
67 }
68 }
```


CategoriaController

Se actualiza el controller para que pueda guardar, eliminar y modificar.

```
CategoriaController.java [-/M] x
Source History
private final CategoriaService categoriaService;
private final MessageSource messageSource;

public CategoriaController(CategoriaService categoriaService, MessageSource messageSource) {
    this.categoriaService = categoriaService;
    this.messageSource = messageSource;
}

@GetMapping("/listado")
public String listado(Model model) {...6 lines }

@PostMapping("/guardar")
public String guardar(@Valid Categoria categoria, @RequestParam MultipartFile imagenFile, RedirectAttributes redirectAttributes) {

    categoriaService.save(categoria, imagenFile);
    redirectAttributes.addFlashAttribute("todoOk", messageSource.getMessage("mensaje.actualizado", null, Locale.getDefault()));

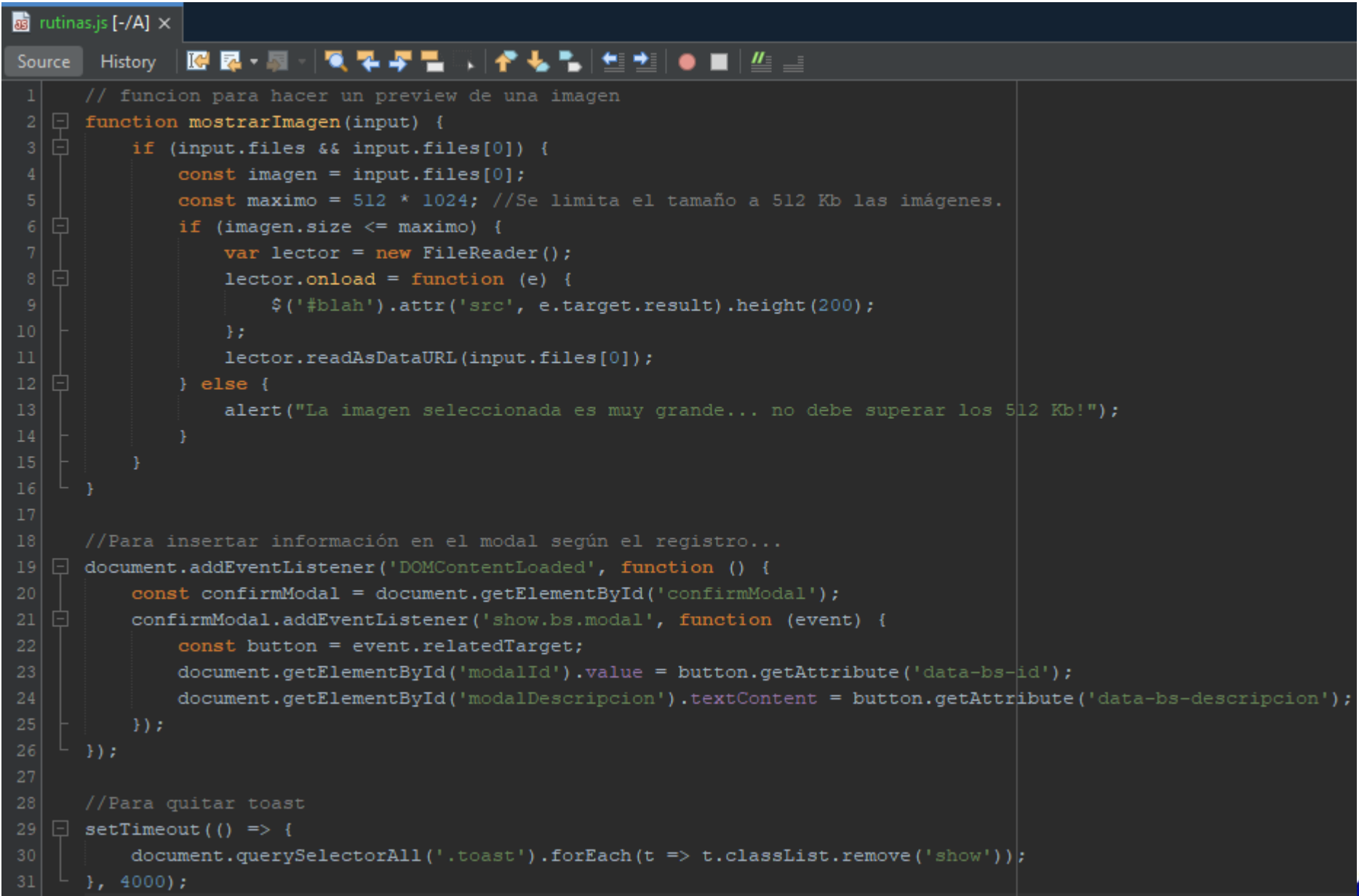
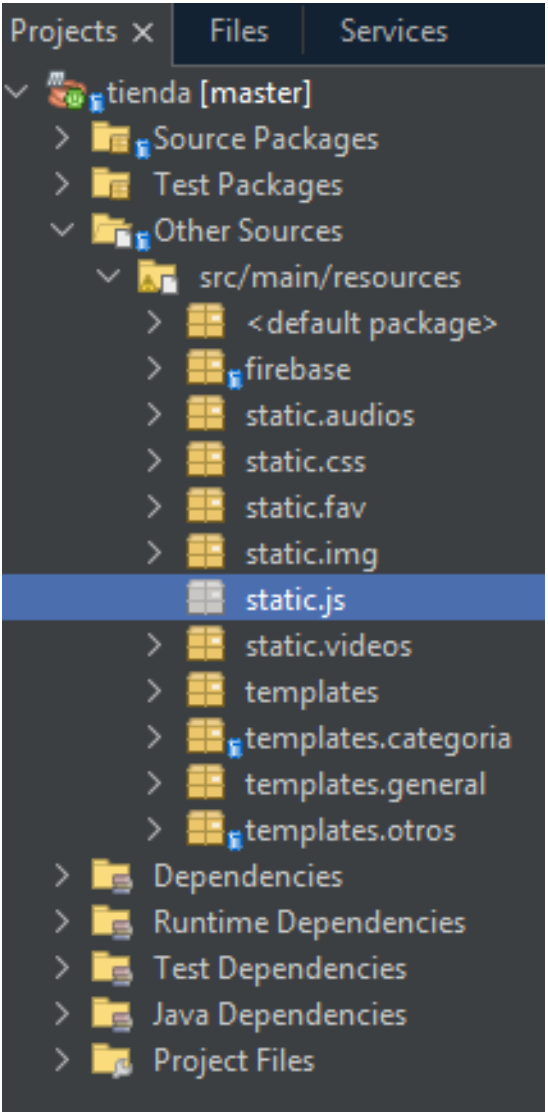
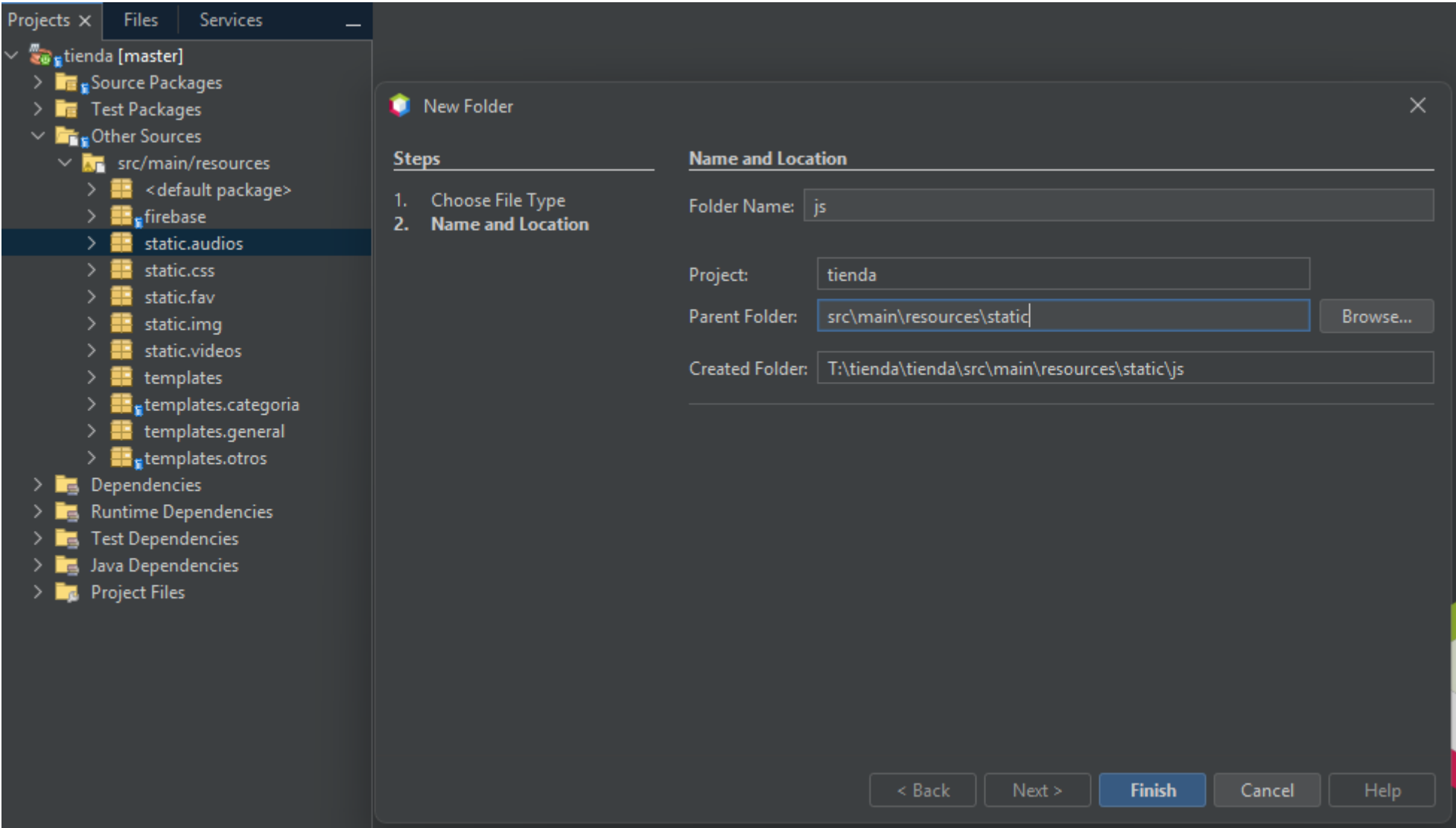
    return "redirect:/categoria/listado";
}

@PostMapping("/eliminar")
public String eliminar(@RequestParam Integer idCategoria, RedirectAttributes redirectAttributes) {
    String titulo = "todoOk";
    String detalle = "mensaje.eliminado";
    try {
        categoriaService.delete(idCategoria);
    } catch (IllegalArgumentException e) {
        titulo = "error"; // Captura la excepción de argumento inválido para el mensaje de "no existe"
        detalle = "categoria.error01";
    } catch (IllegalStateException e) {
        titulo = "error"; // Captura la excepción de estado ilegal para el mensaje de "datos asociados"
        detalle = "categoria.error02";
    } catch (Exception e) {
        titulo = "error"; // Captura cualquier otra excepción inesperada
        detalle = "categoria.error03";
    }
    redirectAttributes.addFlashAttribute(titulo, messageSource.getMessage(detalle, null, Locale.getDefault()));
    return "redirect:/categoria/listado";
}

@GetMapping("/modificar/{idCategoria}")
public String modificar(@PathVariable("idCategoria") Integer idCategoria, Model model, RedirectAttributes redirectAttributes) {
    Optional<Categoria> categoriaOpt = categoriaService.getCategoria(idCategoria);
    if (categoriaOpt.isEmpty()) {
        redirectAttributes.addFlashAttribute("error", messageSource.getMessage("categoria.error01", null, Locale.getDefault()));
        return "redirect:/categoria/listado";
    }
    model.addAttribute("categoria", categoriaOpt.get());
    return "/categoria/modifica";
}
```

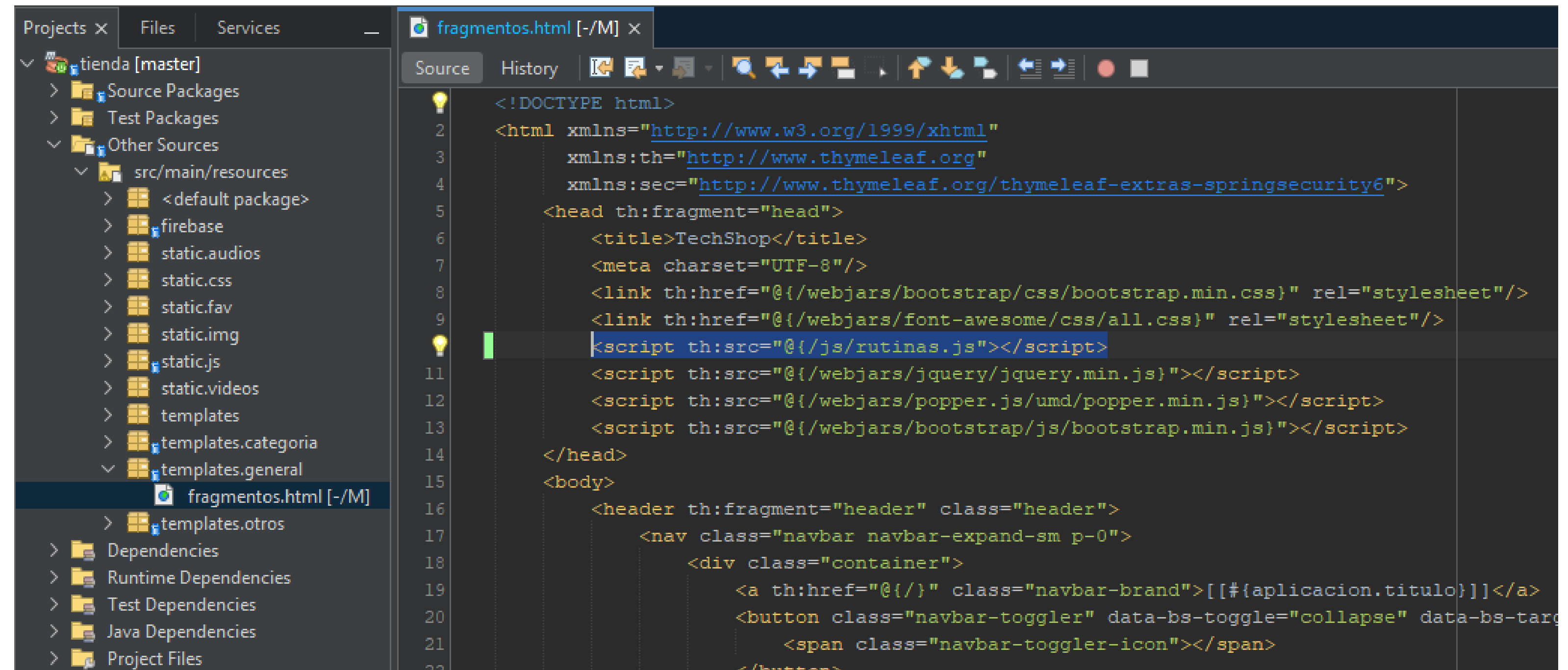
Creacion del folder js en static

Acá colocaremos un archivo de sentencias javascript que llamaremos rutinas.js este archivo se encuentra en los recursos.



fragmentos.html

Se incluye un enlace en fragmentos del folder general



The screenshot shows an IDE with two panels. The left panel displays the project structure for 'tienda [master]'. The right panel shows the source code of 'fragmentos.html'.

Project Structure (Left Panel):

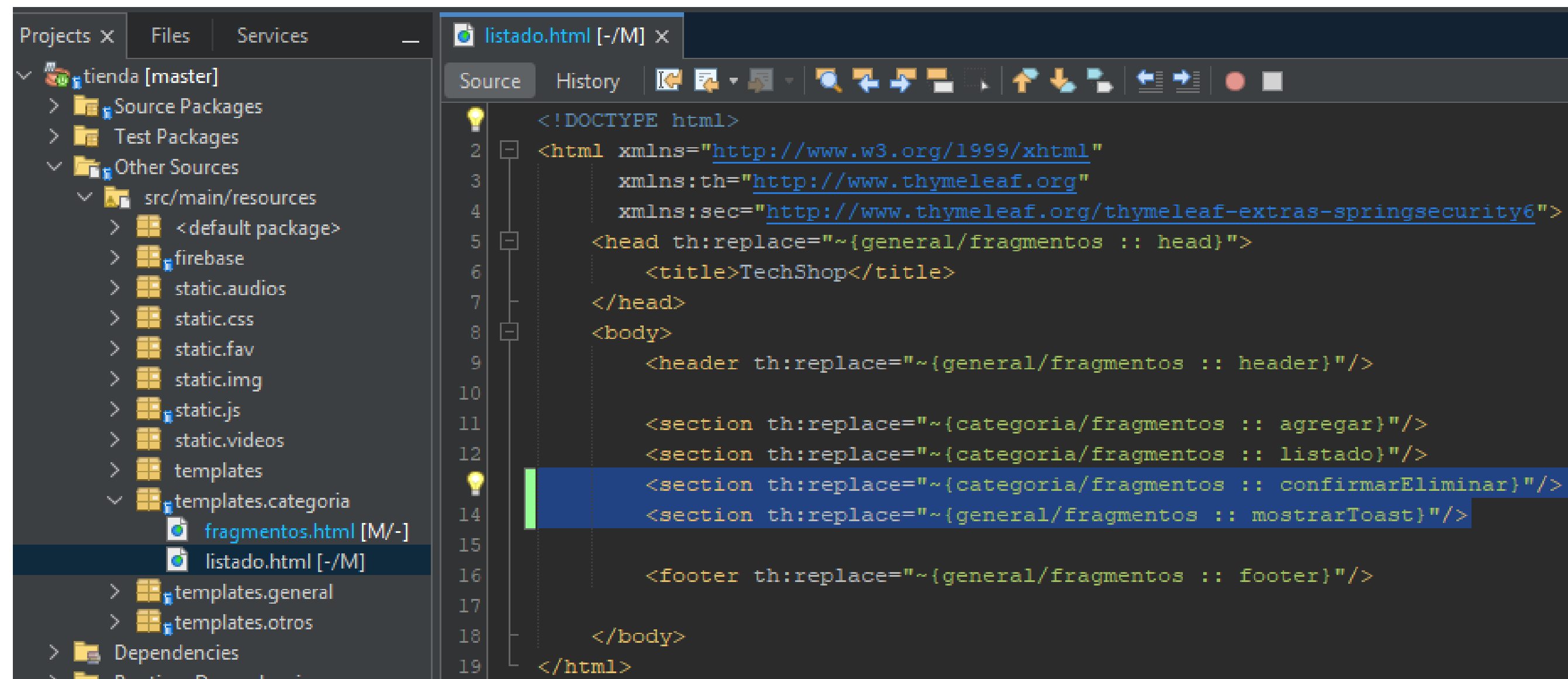
- tienda [master]
 - Source Packages
 - Test Packages
 - Other Sources
 - src/main/resources
 - <default package>
 - firebase
 - static.audios
 - static.css
 - static.fav
 - static.img
 - static.js
 - static.videos
 - templates
 - templates.categoria
 - templates.general
 - fragmentos.html [-/M]**
 - templates.otros
 - Dependencies
 - Runtime Dependencies
 - Test Dependencies
 - Java Dependencies
 - Project Files

Source Code (Right Panel):

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
  <head th:fragment="head">
    <title>TechShop</title>
    <meta charset="UTF-8"/>
    <link th:href="@{/webjars/bootstrap/css/bootstrap.min.css}" rel="stylesheet"/>
    <link th:href="@{/webjars/font-awesome/css/all.css}" rel="stylesheet"/>
    <script th:src="@{/js/rutinas.js}"></script>
    <script th:src="@{/webjars/jquery/jquery.min.js}"></script>
    <script th:src="@{/webjars/popper.js/umd/popper.min.js}"></script>
    <script th:src="@{/webjars/bootstrap/js/bootstrap.min.js}"></script>
  </head>
  <body>
    <header th:fragment="header" class="header">
      <nav class="navbar navbar-expand-sm p-0">
        <div class="container">
          <a th:href="@{/}" class="navbar-brand">[[#{aplicacion.titulo}]]</a>
          <button class="navbar-toggler" data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"
            <span class="navbar-toggler-icon"></span>
          </button>
        </div>
      </nav>
    </header>
  </body>
</html>
```


listado.html

Se crean unas líneas en el archivo listado.html



```
<!-- tienda [master] -->
Source
History
listado.html [-/M] x

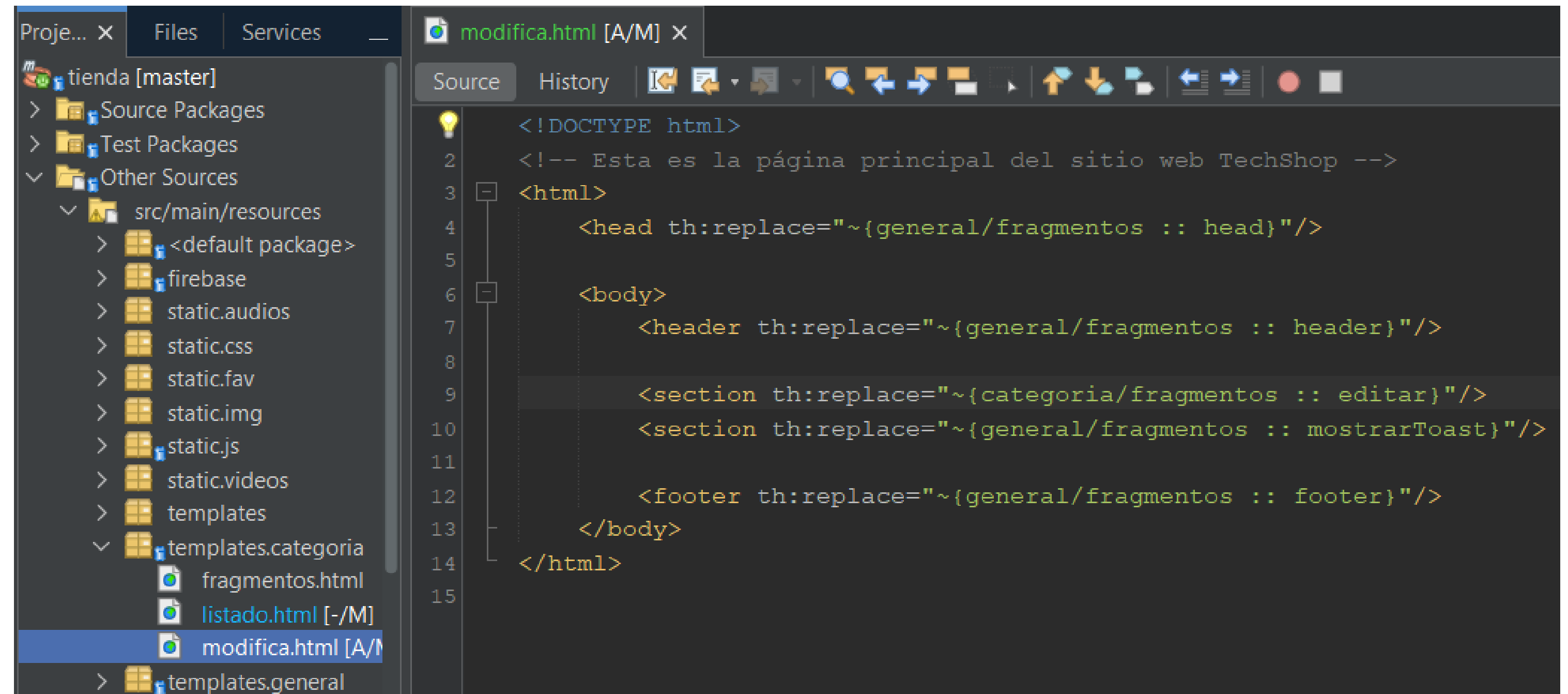
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
  <head th:replace="~{general/fragmentos :: head}">
    <title>TechShop</title>
  </head>
  <body>
    <header th:replace="~{general/fragmentos :: header}" />

    <section th:replace="~{categoria/fragmentos :: agregar}" />
    <section th:replace="~{categoria/fragmentos :: listado}" />
    <section th:replace="~{categoria/fragmentos :: confirmarEliminar}" />
    <section th:replace="~{general/fragmentos :: mostrarToast}" />

    <footer th:replace="~{general/fragmentos :: footer}" />
  </body>
</html>
```

Se crea la página modifica.html

Que mostrará la categoría a modificar, se puede tomar como base el archivo modifica.html

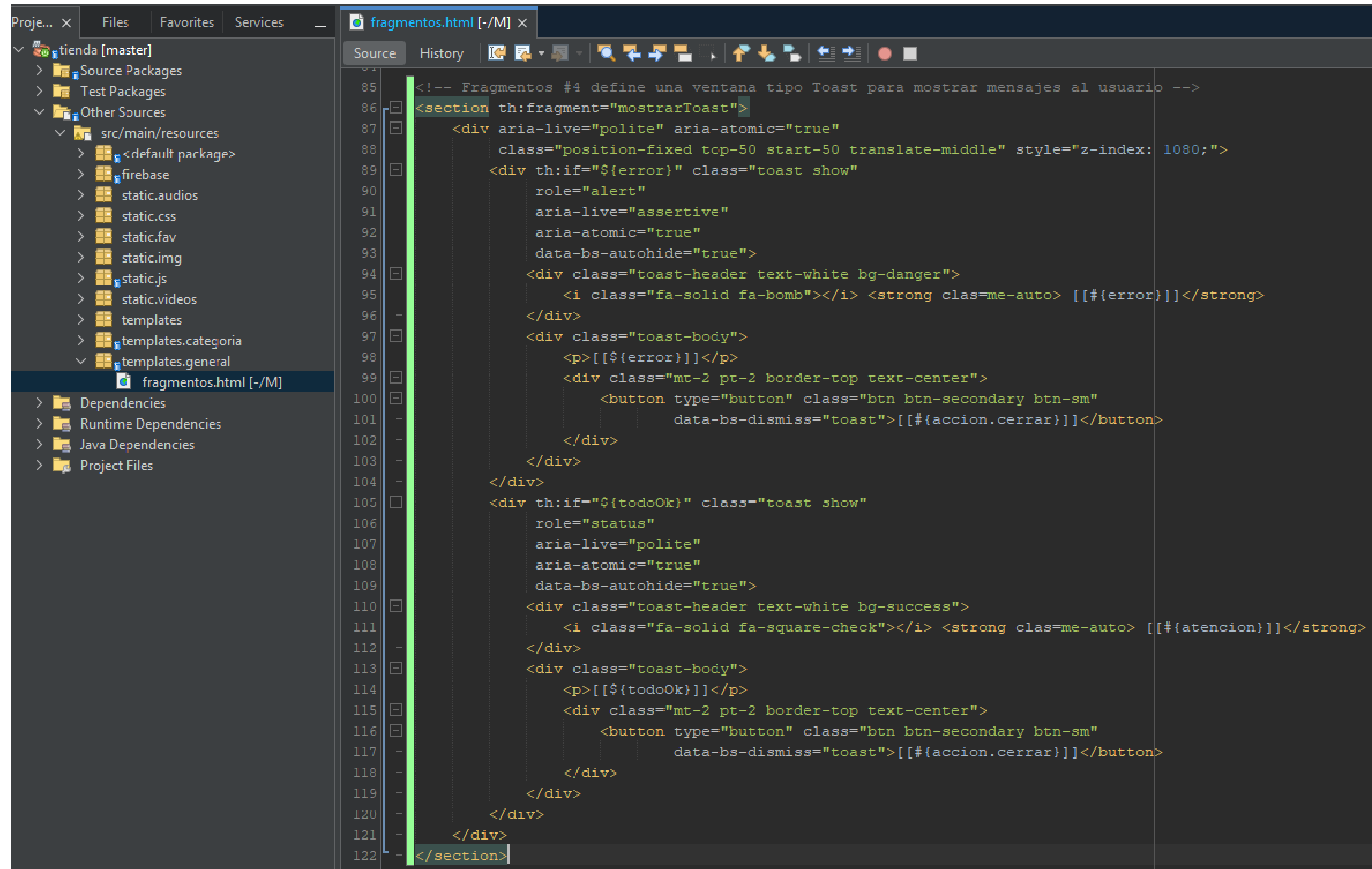


The screenshot shows an IDE interface. On the left, the 'Files' pane displays a project structure for 'tienda [master]'. The 'src/main/resources' directory is expanded, showing various static files and a 'templates' directory. The 'templates' directory is further expanded, showing 'categoria', 'fragmentos.html', 'listado.html [-/M]', and 'modifica.html [A/M]'. The 'modifica.html' file is selected. On the right, the 'Source' pane shows the content of 'modifica.html'. The code is an HTML template using Thymeleaf syntax. It includes a DOCTYPE declaration, a comment indicating it's the main page of the TechShop website, and a structure with a head, body, and footer. The body contains a header, a section for editing a category, a section for showing a toast message, and a footer. The code uses Thymeleaf's 'th:replace' attribute to include fragments from other templates.

```
<!DOCTYPE html>
<!-- Esta es la página principal del sitio web TechShop -->
<html>
  <head th:replace="~{general/fragmentos :: head}"/>
  <body>
    <header th:replace="~{general/fragmentos :: header}"/>
    <section th:replace="~{categoria/fragmentos :: editar}"/>
    <section th:replace="~{general/fragmentos :: mostrarToast}"/>
    <footer th:replace="~{general/fragmentos :: footer}"/>
  </body>
</html>
```

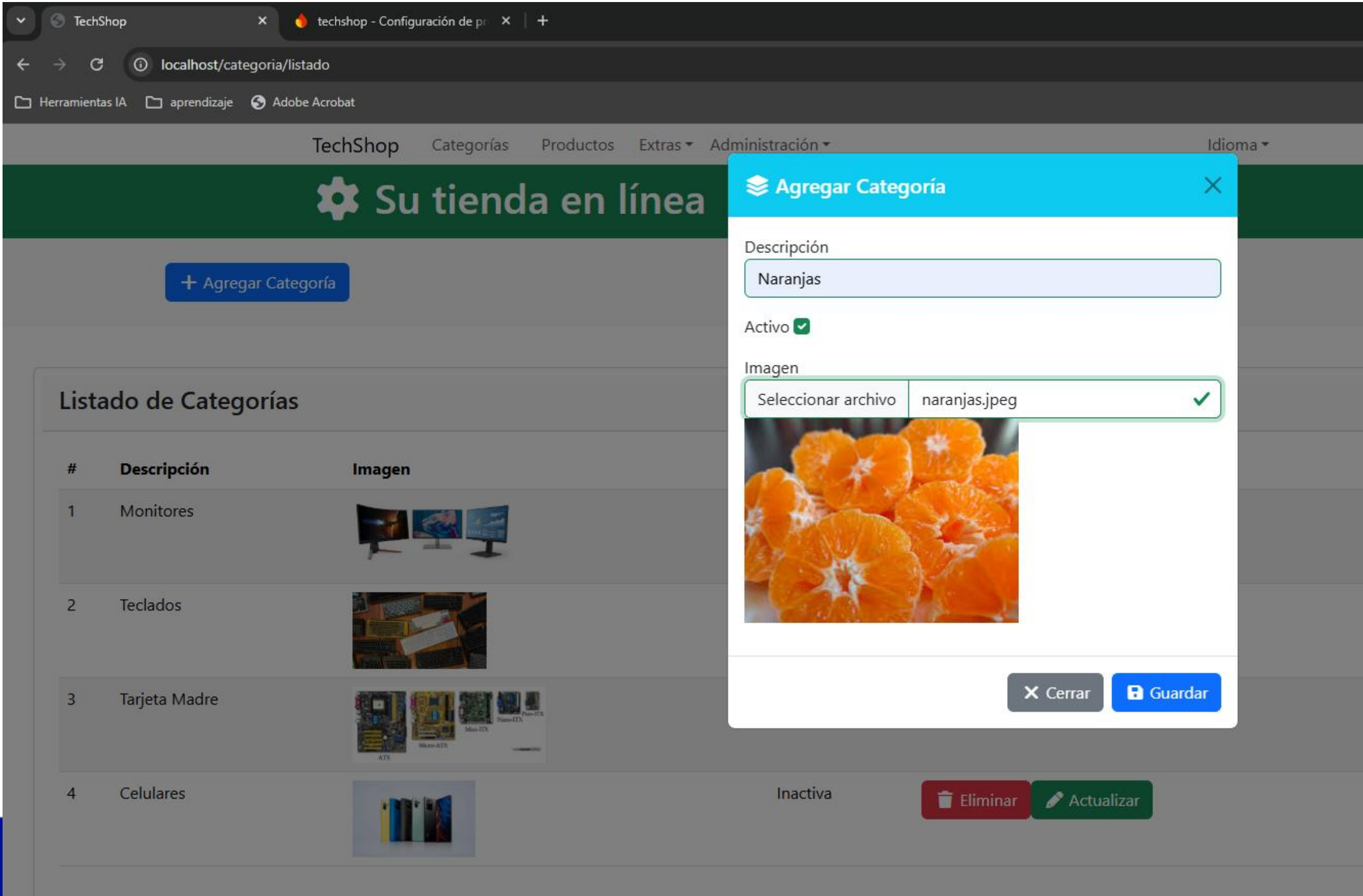
fragmentos.html

Se crea un fragment en general\fragmentos, se coloca la parte del mostrarToast



The image shows a screenshot of an IDE with two panels. The left panel is the 'Project Explorer' showing a project named 'tienda [master]'. It has a tree structure with 'Source Packages', 'Test Packages', and 'Other Sources'. Under 'Other Sources', there is a folder 'src/main/resources' containing various static files like 'static.audios', 'static.css', 'static.fav', 'static.img', 'static.js', 'static.videos', 'templates', 'templates.categoria', and 'templates.general'. The 'fragmentos.html [-/M]' file is selected under 'templates.general'. The right panel shows the content of 'fragmentos.html' in the 'Source' tab. The code is a Thymeleaf fragment defining a toast message. It starts with a comment: '

Agregando una categoría



Modificando una categoria

TechShop

techshop - Configuración de pr

localhost/categoria/modificar

Herramientas IAaprendizajeAdobe Acrobat

TechShopCategoríasProductosExtrasAdministraciónIdioma

Su tienda en línea

← Regresar

✓ Guardar

Actualizar

Descripción

Naranjas

✓

Activo

✓

Imagen

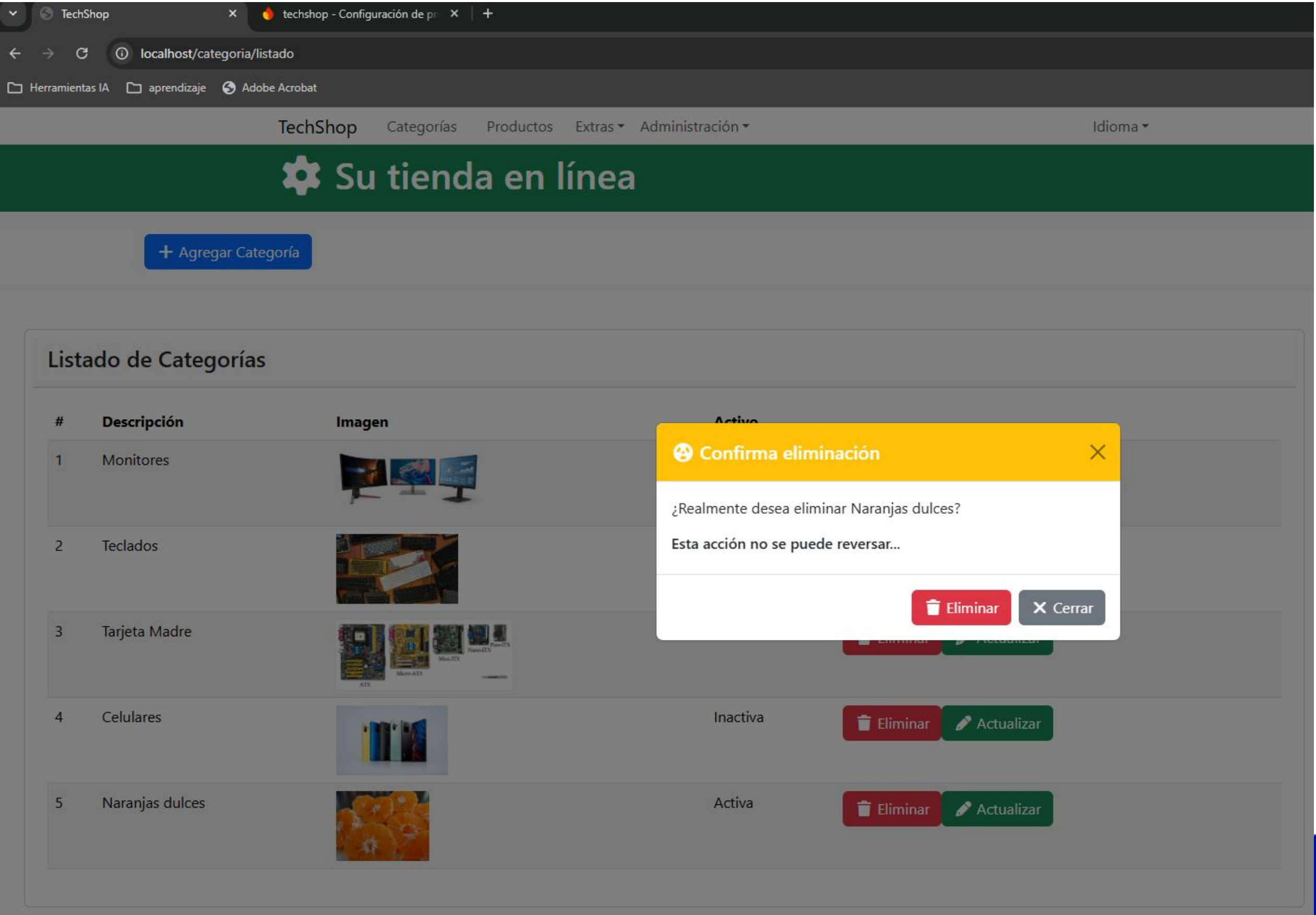
Seleccionar archivo

Sin archivos seleccionados

✓



Eliminando una categoría



Nos vemos en semana 5

Reto: intenta modificar el esquema de colores que tienen las ventanas, hazlo a tu gusto, cambia el tiempo predeterminado para que se quite la ventana Toast que aparece.

Tip: investiga como usar colores personalizados, revisa cómo cambiar el timer del Toast